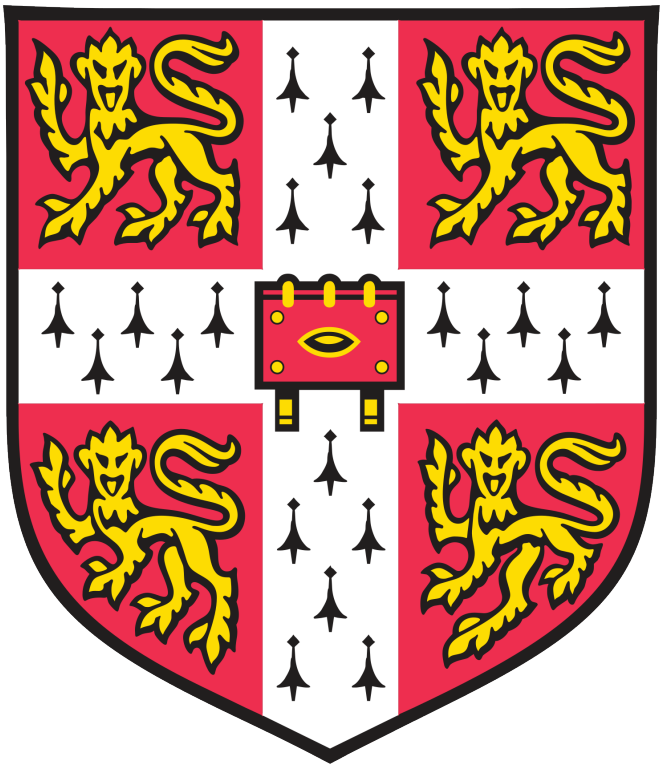
7932Z

WhisperWise: Emergent Communication in Multi-Agent Reinforcement Learning for Complex Tasks



Computer Science Tripos – Part II

Queens' College

2026

Declaration

I, the candidate for Part II of the Computer Science Tripos with Blind Grading Number 7932Z, hereby declare that this report and the work described in it are my own work, unaided except as may be specified below, and that the report does not contain material that has already been used to any substantial extent for a comparable purpose. In preparation of this report, I adhered to the Department of Computer Science and Technology AI Policy. I am content for my report to be made available to the students and staff of the University.

Signed: 7932Z
Date: May 13, 2026

Proforma

Candidate number:	7932Z
Project Title:	**WhisperWise: Emergent Communication in Multi-Agent Reinforcement Learning for Complex Tasks**
Examination:	**Computer Science Tripos – Part II, 2026**
Word Count:	11,954[1]
Code Line Count:	13,722[2]
Project Originator:	The Candidate
Project Supervisor:	Ian Lewis

Original Aims of the Project

The aim of this project was to determine whether agents can develop their own language to communicate effectively and solve cooperative tasks. The project aimed to compare a specific architecture (AI Mother Tongue) with baseline scenarios in which agents cannot communicate or use a basic, fixed communication system. Success was measured by how well the agents completed their tasks and whether their generated messages carried understandable, contextual meaning.

Work Completed

A scalable simulation and multi-agent reinforcement learning controller were built to evaluate four communication strategies. Evaluations revealed that agents developed compressed, context-aware languages. However, whilst the AI Mother Tongue architecture enabled meaningful communication, its structural bottleneck impeded the agents' learning process. In contrast, a novel protocol, WhisperWise, outperformed AI Mother Tongue by $\sim 40\%$ on the basic task and scaled to more demanding tasks with fewer training steps than AI Mother Tongue.

Special Difficulties

None.

[1] This word count was computed using `texcount`.

[2] This code line count was computed with a custom script using Unix built-in tool `wc`.

Contents

Chapter 1

Introduction

Since DeepMind's AlphaGo in 2016 [1], interest in reinforcement learning has steadily increased [2], leading to breakthroughs in finance, trading, robotics, and even video games. A clear next step for reinforcement learning is multi-agent reinforcement learning, in which multiple agents are placed in the same environment and must learn to interact efficiently with one another. This research has the potential to improve surgery robots, traffic optimisations and search-and-rescue operations.

However, several challenges make multi-agent reinforcement learning more difficult in practice, such as non-stationarity, credit assignment issues (both explained in Section 2.2), and the one I will focus on in this project: communication problems [3]. Unlike single-agent reinforcement learning, multi-agent reinforcement learning can leverage communication to improve agents' knowledge of their environment and/or guide them toward the optimal next step. In Partially Observable Markov Decision Processes (POMDP), communication is required for agents to solve the problem [4].

Traditionally, communication is facilitated through every agent broadcasting raw continuous values to every other agent. While this is perfect for backpropagation, it scales poorly due to limited bandwidth between agents and is difficult for humans to interpret.

Replacing these continuous communication vectors with discrete tokens is highly desirable for both bandwidth and interoperability. However, learning a discrete language on-policy suffers from the "Joint Exploration Problem" [5], in which agents get stuck in a local optimum by learning to completely ignore the communication channel, which initially consists solely of noise. This local optimum is named the "Communication Vacuum Equilibrium" [5].

This dissertation focuses on the paper "AI Mother Tongue: Self-Emergent Communication in MARL via Endogenous Symbol Systems" [6]. In this paper, Liu [6] proposes abandoning learning this discrete language during the reinforcement learning phase of training. Instead, they propose using a pre-trained discrete language model acting as a semantic bottleneck.

By using a Vector-Quantised Variational Autoencoder (VQ-VAE), continuous observations can be mapped to a finite set of discrete symbols. Liu [6] then showed that this autoencoder can be frozen during training, and the agents can be taught semantic meaning through reflection strategies - custom loss functions that force agents to understand contextual meaning and intent of these symbols. This methodology allows communication protocols to evolve naturally without implementing hard biases [5].

Whilst the AI Mother Tongue [6] paper suggests an interesting novel idea, it only evaluates this on relatively simple cooperative tasks. To ascertain whether this architecture is effective and robust enough for practical use, it needs rigorous evaluation within a more complex environment where agents possess asymmetric information and must cooperate to discover hidden stations. This dissertation provides an extensive evaluation and extends the ideas discussed in the paper.

Research Question: To what extent does the AI Mother Tongue architecture enable emergent discrete communication in complex multi-agent POMDPs when scaled to high-throughput Multi-Agent Proximal Policy Optimisation (MAPPO), relative to no-communication and fixed discrete communication baselines?

1.1 Project Overview & Contributions

To answer the proposed research question, this project implements a general framework for emergent communication in multi-agent systems using a configurable assembly-line-style environment. It is designed to be a reusable toolkit for training and analysing communication protocols in POMDPs. Users can interact with the framework through a single script with modes for language pretraining, imitation learning and reinforcement learning.

The main contributions in this project are:

A configurable high-throughput multi-agent reinforcement learning (MARL) environment - a vectorised, Rust-based simulation was implemented that simulates a discrete two-dimensional environment, supporting hundreds of parallel worlds and a real-time, lightweight visualisation.

A MAPPO algorithm with hot-swappable communication protocols - a MAPPO-based MARL training script under a centralised training, decentralised execution framework was engineered. It used a shared actor, a global critic, a Long Short-Term Memory (LSTM) module, a custom rollout buffer for efficient storage and Generalised Advantage Estimation (GAE) calculations, and a communication protocol interface to allow hot-swapping of protocols.

The AI Mother Tongue architecture - a communication protocol that follows and extends the AI Mother Tongue architecture [6] was built. Pretrained VQ-VAE or stochastic (SQ-VAE) and hierarchical (HQ-VAE) variants were injected into the actor network and frozen before MARL training.

The WhisperWise Protocol - a communication protocol that uses reflection strategies to teach agents how to use the communication channels, overcoming the "Joint Exploration Problem" [5].

Quantitative evaluation of communication protocols - communication protocols, including AI Mother Tongue, Discrete Communication and No Communication, were evaluated using metrics such as task reward.

Semantic analysis of communication and memory - conditional distributions such as $P(\text{Target}^{t}_{\text{partner}}|\text{Code}^{t}_{self})$, $P(\text{Action}^{t+1}_{\text{partner}}|Code^{t}_{self})$ were collected, visualised and qualitatively analysed.

Chapter 2

Preparation

This chapter defines the cooperative environment and formalises the MARL problem used to study emergent learning. It reviews relevant background material and motivates the choice of MAPPO as the basis for the project. Finally, it translates the research question into concrete success criteria and design requirements to guide Chapter 3.

2.1 Environment

To evaluate communication in this framework, we investigate the following cooperative game.

There is an MxN board. The game lasts for T timesteps, and at each timestep, every agent simultaneously completes one action. The goal of the game is for the agents to cooperate and complete a recipe.

The world contains entities called agents and stations.

For each world generated, stations are randomly placed across the map and are immovable. This randomisation prevents agents from overfitting a single layout. Each station has an assigned ingredient. The station can either be of type 'pick up' or 'drop off'. A 'pick up' station only allows the agents to take ingredients from it. A 'drop off' station only allows agents to drop off the assigned ingredient.

For a similar reason, agents are randomly placed across the map at world initialisation.

Each agent is assigned an ordered pair of ingredient sets (I_{in}, I_{out}). The agent must first collect all ingredients in I_{in}. Then the agent's inventory is replaced with I_{out}, and the agent must remove all the ingredients from their inventory. This defines a single recipe, but agents may complete multiple recipes in their lifetime.

Ingredients can be collected and dropped off at both other agents and stations.

One important feature of this game is that the agents have a limited field of view and can only see the nearest entity, excluding their own pick-up station, at any given timestep. This converts this game from a Markov Decision Process (MDP) to a Partially Observable Markov Decision Process (POMDP) [4]. The agents must work together to discover the required stations and use them to complete recipes.

At each timestep, every agent takes a single action. Agents have five possible actions: move north, south, east, west, and interact. The first four of which move the agent one step in the given direction. If agent A executes the 'Interact' action:

1. If visible agent B has a required item, place it in A's inventory, and remove it from B's inventory.
2. If visible 'pick up' station has a required item, place in A's inventory.
3. If visible 'drop off' station wants an item we have, remove from A's inventory.

The five actions form the action space $\mathcal{A}$ in the POMDP definition (Section 2.2).

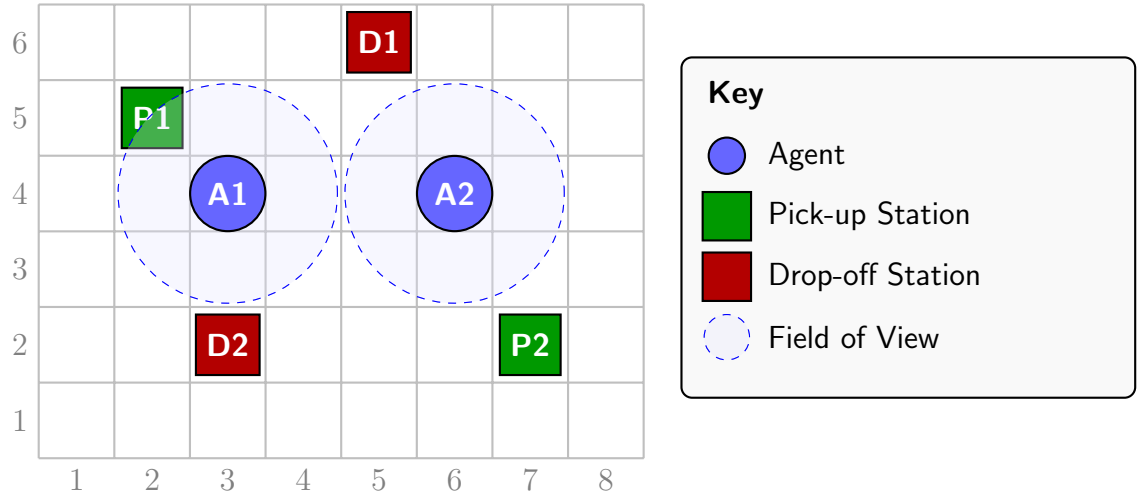


Figure 2.1: Example World.

A typical world is shown in Figure 2.1. There are two agents and two stations. Each agent's own pick-up station is visible only to the other if within the limited field of view. If no entity is within the agent's field of view, then the agent views a 'no-entity' token. Without communication, both agents would have to search the map independently until they discovered the station. With communication, the agents can tell one another the location of the required pick-up station. This shows how the environment creates asymmetric information and how communication can solve it.

2.2 Multi-Agent Reinforcement Learning Background

Reinforcement Learning (RL) differs from classical supervised and unsupervised learning because the data is not obtained before training. Instead, environments must be built that allow agents to take actions and receive feedback from those actions. The agents must learn to maximise the total reward collected over time.

Classic RL is modelled as a Markov Decision Process, but to investigate communication, each agent's information must be limited, forcing them to communicate missing information to one another. Therefore, this RL environment is modelled as a Partially Observable Markov Decision Process (POMDP).

A POMDP is formally defined as $\langle \mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \Omega, \mathcal{O}, \gamma \rangle$. Where:

- $\mathcal{S}$ is the set of all possible states of the environment.
- $\mathcal{A}$ is the action space. {North, South, East, West, Interact}.
- $\mathcal{P}$ is the transition probability function $P(s'|s, a)$, representing the probability that taking action a in state s will lead to state s'.
- $\mathcal{R}$ is the reward function. See Section 3.3.1.

- Ω is the set of all possible local observations.
- $\mathcal{O}$ is the observation function $P(o|s, a)$, representing the agent seeing observation o given state s and previous action was a.
- γ is the discount factor. Set at 0.99 following [7].

For the environment described in Section 2.1, $\mathcal{S}$ consists of the world layout including the entities' locations, the agents' inventories, the station type and the station's resources. Ω contains each agent's nearest entity if within the range of view, communication codes all other agents emitted in the previous timestep, and their own state (i.e. location and inventory). Therefore, the system is a cooperative Decentralised Partially Observable Markov Decision Process (Dec-POMDP) in which multiple agents act on the same state, receive individual local observations, and share a common reward signal [8].

This dissertation's focus is emergent communication in MARL; however, MARL is often subject to various challenges that must first be overcome before the communication can be analysed. One such problem is non-stationarity, which arises when multiple agents learn simultaneously, creating an environment that changes unpredictably from an individual agent's perspective. Another challenge is the credit assignment problem, which arises when multiple agents work cooperatively toward a single goal, making it difficult to determine each agent's contribution [9]. To mitigate these problems, an appropriate, well-tested algorithm is chosen.

Three candidate approaches were considered (QMIX, MADDPG, and MAPPO): These algorithms are widely used as reference baselines in contemporary MARL benchmark suites, allowing my results to be directly comparable to prior work [10].

Value-Decomposition Networks (VDN) [11], such as QMIX [12], are unsuitable for the given game. QMIX relies on a monotonicity assumption (the global optimal action-value is monotonic for each agent's action-values). For training communication, the utility of a code broadcast by agents depends on the environment state, which directly violates the monotonicity assumption.

Off-policy actor-critic methods such as Multi-Agent Deep Deterministic Policy Gradient (MADDPG) [13] are also unsuitable, as code broadcast semantically shifts during training, leading to new policies being trained on stale data.

Multi-Agent Proximal Policy Optimisation (MAPPO) [14] is an extension of Proximal Policy Optimisation (PPO) [15] to cooperative multi-agent settings, itself derived from Trust Region Policy Optimisation (TRPO)[16]. MAPPO shares a policy across all agents and uses a centralised value function with access to the global state during training, whilst each agent acts in a decentralised manner at execution time. This centralised training, decentralised execution (CTDE) [17] paradigm is well-suited for emergent communication in cooperative MARL, as it leverages global information during training while preserving decentralised execution. The centralised critic during training helps mitigate non-stationarity and credit assignment problems, separating them from communication problems by learning from joint states and rewards [18], whilst PPO-style clipped updates provide stable policy optimisation. All agents sharing a single policy but acting independently at execution time ensure that any performance gains arise from useful information in the communication channel rather than from hidden, centralised information.

MAPPO is chosen as the approach to focus on emergent communication and mitigate common MARL problems (non-stationarity, credit assignment).

MAPPO optimises two main components: each agent's Policy (Actor) π_{θ_i} and the Value Function (Critic) V_ϕ.

Actor: Policy Optimisation

The policy network is parameterised by θ whose goal is to maximise the clipped surrogate objective, preventing drastic changes to the policy from single timesteps. For every agent i, define the probability ratio $r_i(\theta)$ as:

$$r_i(\theta) = \frac{\pi_\theta(a_i|o_i)}{\pi_{\theta_{old}}(a_i|o_i)} \tag{2.1}$$

The MAPPO objective function is:

$$L(\theta) = \mathbb{E}_i \left[\min \left(r_i(\theta)\hat{A}_i,\ \text{clip}(r_i(\theta), 1-\epsilon, 1+\epsilon)\hat{A}_i \right) \right] \tag{2.2}$$

Where:

- a_i is the action taken by agent i.
- o_i is the local observation of agent i.
- $\pi_\theta(a_i|o_i)$ is the current policy's probability of taking action a_i given observation o_i.
- $\pi_{\theta_{old}}(a_i|o_i)$ is the old policy's probability of taking the same action before the update.
- $\mathbb{E}_i$ denotes the empirical expectation over a batch of agent experiences.
- $\hat{A}_i$ is the Advantage Estimation for agent i (detailed later in Section 2.2).
- ϵ is a hyperparameter that limits the policy update.

Critic: Value Function Learning

Unlike standard PPO, the Value Function is not per-agent, but per-world. It has access to global information. The critic network is parameterised by ϕ whose goal is to minimise the Mean Squared Error between its own prediction and the discounted returns R_t.

$$L(\phi) = \mathbb{E}_t \left[(V_\phi(S_t) - R_t)^2 \right] \tag{2.3}$$

Where:

- $\mathbb{E}_t$ denotes the expectation over timesteps t within the sampled trajectories.
- S_t is the global state of the environment at timestep t.
- $V_\phi(S_t)$ is the critic network's estimate of the value of the global state S_t.

Generalised Advantage Estimation

MAPPO uses Generalised Advantage Estimation [19, 20] to balance bias and variance. To calculate, we first find the Temporal Difference (TD) Residual (representing the error in our current estimate of the state value):

$$\delta_t = r_t + \gamma V(s_{t+1}) - V(s_t) \tag{2.4}$$

We then take an exponentially weighted moving average of our estimated advantage over different time horizons.

$$\hat{A}_t = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{t+l} \tag{2.5}$$

Where:

- δ_t is the TD residual at timestep t.
- r_t is the immediate reward received at timestep t.
- λ is the GAE trace-decay parameter ($0 \leq \lambda \leq 1$) that smooths the bias-variance trade-off.
- l is the forward-looking step index.

2.3 Emergent Communication in Multi-Agent Reinforcement Learning

Early work in emergent communication studied how agents can learn to communicate with one another without concern for interoperability or bandwidth, typically in two-agent coordination tasks [21]. This established that a communication channel can emerge purely from task reward, but they often used continuous communication.

Foundational protocols such as Differentiable Inter-Agent Learning (DIAL) [21] enabled continuous gradient flow between agents during training. This relied on passing high-dimensional, floating-point vectors, which in reality is unreasonable due to limited bandwidth. Later work, such as Targeted Multi-Agent Communication (TarMAC) [22], attempted to address this bandwidth issue by employing attention mechanisms to learn which agent to communicate with. However, TarMAC still relied on passing high-dimensional, floating-point vectors, which remain neither realistic nor interpretable.

More recent research in multi-agent reinforcement learning studies agents learning to send discrete symbols, showing that discrete channels can match or outperform continuous channels, whilst also allowing for interoperability [23]. With this interoperability, the trust deficit in safety-critical applications can be reduced and even allow human-in-the-loop settings.

To understand how communication improves performance, the agents' uncertainty is formalised. Because agents cannot observe the entire global state, each agent has a belief state $b(s)$ that represents a probability distribution over the latent variables [24]. This can be formalised using Shannon Entropy [25]:

$$H(b) = -\sum_{s \in S} b(s) \log b(s) \tag{2.6}$$

A high $H(b)$ indicates high uncertainty, which injects high variance into the policy gradient estimates.

Communication mitigates this. If messages are informative about the latent state, then the mutual information between the state and the communication token is non-negative ($I(S;C) \geq 0$), giving a lower bound on $H(S|C) \leq H(S)$. High uncertainty of the latent state leads to noisier estimates of the value function. The variance of the policy gradient

can be lowered by reducing $H(b)$, which often leads to more stable PPO updates within the trust-region constraint. This formalises why communication protocols converge faster and more stably than non-communicative baselines.

However, a challenge in learning discrete communication on-policy is the "Joint Exploration Problem" and the resulting "Communication Vacuum Equilibrium", as stated by Eccles et al [5]. At the start of training, the agent's communication policy outputs random values, providing no predictive value. Therefore, the section of the agent's network which listens to this noise rapidly updates its weights to silence it. However, once the weights become zero, the gradients stop flowing through the communication channel, trapping the agents in the Communication Vacuum Equilibrium.

Several works attempt to address this Communication Vacuum Equilibrium by introducing architectural biases or auxiliary objectives (e.g., adding rewards for mutual information between messages and observations). Whilst these approaches improve communication usage, they reduce generality. Liu [6] proposes the AI Mother Tongue architecture, in which language is learned offline, separate from online reinforcement learning. Then the agents are trained in this language using auxiliary losses rather than relying on architectural biases or auxiliary objectives. This project adopts this architecture and expands both the evaluation and task to investigate under what conditions communication emerges and how it impacts performance.

2.4 AI Mother Tongue Architecture

The AI Mother Tongue architecture addresses the "Joint Exploration Problem" by separating the emergence of discrete symbols from the reinforcement learning training and adding auxiliary losses as described in Figure 2.2. Liu reports promising phenomena such as sudden semantic compression and interpretable symbol usage, but their evaluation is restricted to a simple cooperative task [6]. The underlying discrete bottleneck uses Vector-Quantised Variational Autoencoders (VQ-VAE) [26]. Still, Liu proposes more expressive codebook learning methods, such as Stochastically-Quantised Variational Autoencoders (SQ-VAE) [27], and ways to advance the language, such as using Hierarchically-Quantised Variational Autoencoders (HQ-VAE) [28].

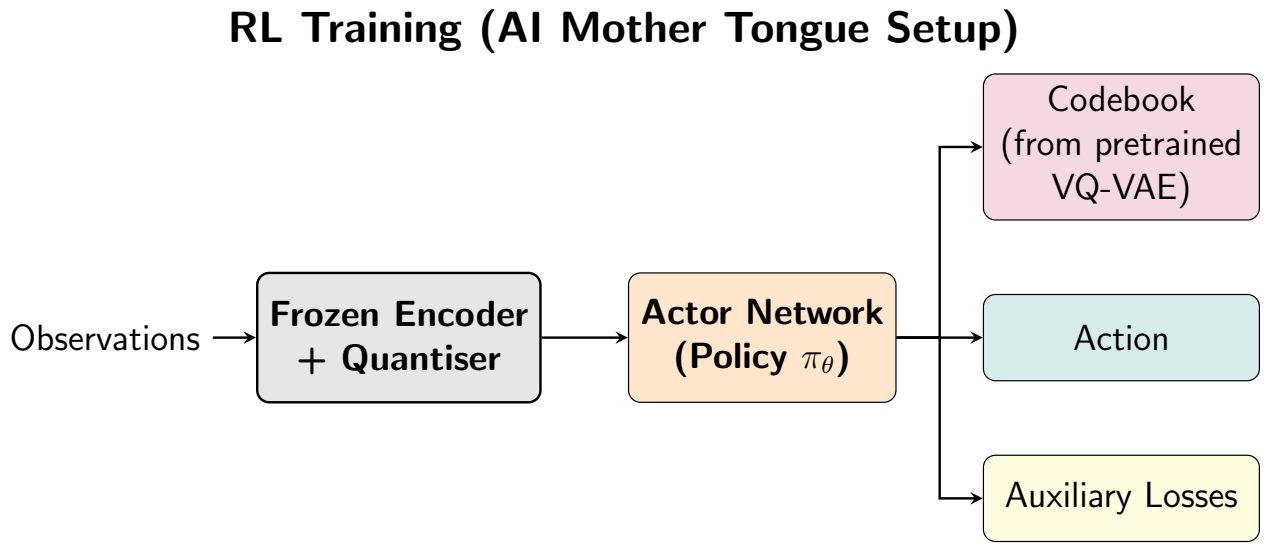


Figure 2.2: Architecture diagram for frozen Vector-Quantised Variational Autoencoder (VQ-VAE) pretraining and RL training the actor network.

The architecture of the AI Mother Tongue protocol proposed by Liu [6] is shown in Figure 2.2. The VAE is first trained offline using collected observations, and the encoder and codebook (i.e., all possible quantised values) are saved. Liu explicitly states values indicating a healthy encoder and codebook (reconstruction loss between 0.08 and 0.15, and commitment loss below 1.0). An evaluation of the health of the trained codebooks is presented in Appendix D.

At the initialisation of the RL phase, the encoder and codebook are loaded, frozen, and used for the actor network's training. The architecture has three outputs:

- **Codebook** A code is emitted from the pretrained codebook chosen by a Multi-Layer Perceptron (MLP).
- **Action Head** Liu uses a simple linear rule as the action head based on the code emitted by the architecture and a pre-selected value K.

$$\text{Action}(code) = \begin{cases} \text{Action 1}, & \text{if code} < \frac{K}{2} \\ \text{Action 2}, & \text{otherwise} \end{cases} \tag{2.7}$$

- **Auxiliary Head** These heads predict the value or likely effect of the emitted codes, helping agents learn which codes are useful and how other agents may respond. The auxiliary heads provide additional learning signals that are backpropagated into the actor network to guide and improve policy optimisation.

This dissertation adopts the AI Mother Tongue architecture but extends it to a more complex MAPPO setting. Instead of a simple linear action head used by Liu, a Multi-Layer Perceptron (MLP) is used. The actor network will also contain a Long Short-Term Memory (LSTM) module. In addition to VQ-VAE, this project evaluates alternative discrete bottlenecks such as SQ-VAE and HQ-VAE (Appendix E).

Liu proposes that the "Joint Exploration Problem" is solved by both semantic bottlenecks and reflection strategies; however, they do not evaluate the effects of either individually. This dissertation hypothesises that the complex semantic bottleneck may, in fact, be impeding the agents' learning, and that applying the reflection strategies to a standard on-policy discrete communication protocol may yield a more efficient novel solution.

2.5 Requirements Analysis

2.5.1 Success Criteria

C1 - High-Throughput MARL environment The system should provide a configurable, vectorised Rust environment with Python bindings, achieving at least 50K environment steps per second and supporting real-time visualisations.

C2 - MARL Framework A functioning multi-agent reinforcement learning implementation with a shared actor and centralised critic capable of learning the complex environment. *VDN-based framework chosen in the project proposal changed to a MAPPO-based framework (see Section 2.2 for justification)*

C3 - Baseline Communication Protocols At least two protocols (no communication and discrete communication) serve as baselines for comparing the AI Mother Tongue architecture.

C4 - AI Mother Tongue Architecture Extend the AI Mother Tongue architecture to a more complex environment. Success will be quantified by a quantitative evaluation of the communication protocols, including achieving a Gini coefficient of at least 0.7, proving a low-entropy active vocabulary and demonstrating a Zipfian distribution in codebook utilisation. Furthermore, the communication must exhibit context-aware signalling, verified with conditional probability distributions (e.g., $P(\text{Target}^t_{\text{partner}}|\text{Code}^t_{self})$).

C5 - Software Engineering and Reproducibility The training and evaluation pipeline should automatically train and evaluate models and allow interchangeability across communication protocols. This pipeline includes high testing coverage, logging, and checkpointing, enabling other researchers to reproduce results.

These criteria directly follow from the central research question from Section 1. C1 and C2 ensure the question is asked in a non-toy setting. Without the high-throughput MARL environment and the stable MARL training, the AI Mother Tongue architecture [6] would not be evaluated under realistic conditions. C1 and C2 also establish a robust, scalable foundation for future MARL research. C3 ensures a comparative evaluation of the AI Mother Tongue architecture against realistic baselines. C4 allows for a fair, in-depth evaluation of the protocol. Finally, C5 encourages testing, configuration, logging and checkpointing to create a reusable toolkit for other researchers to further this work.

2.5.2 MoSCoW Analysis

To prioritise tasks, a MoSCoW analysis was completed.

Table 2.1: MoSCoW Analysis of project requirements and associated risks.

Priority	**Feature**
Must Have	Vectorised Rust environment with PyO3 bindings
	Functioning MAPPO-based multi-agent reinforcement learning system
	At least two communication protocols, including the AI Mother Tongue architecture
Should Have	Real time visualisation of simulations running
	Pipeline to train autoencoders
Could Have	Advanced VAE varients: SQ-VAE, HQ-VAE
	A separate environment task to test agents on
	Hyperparameter tuning for all ML training loops
Won't Have	Non-Stationary Environment
	Modelling of network latency and packet loss

2.6 System Design & Tool Selection

The project's development used iterative sprints focused on incremental value, following the Agile methodology [29]. The requirements for emergent communication in MARL were inherently exploratory. This is due to algorithms evolving in response to evaluations such as codebook utilisation and training success. The Agile methodology proceeds through the phases: Requirements → Planning → Implementation → Testing/Evaluation → Repeat.

Python was chosen as the programming language for the controller primarily because of its strong machine learning and data analysis libraries. (including but not limited to NumPy [30], Pandas [31], Matplotlib [32], and Jupyter Notebooks [33]). PyTorch [34] was chosen to develop the models due to its high performance and imperative execution, which allows rapid iterative prototyping.

However, Python is not a performance-oriented language and, as of this development, has a Global Interpreter Lock (GIL) [35], meaning that simulations of the environment must be executed serially, severely bottlenecking performance. Rust was developed for its high performance and memory safety, enabling rapid development. To enable vectorisation of the environment, `Rayon` [36] was chosen, as it provides safe, data-parallel multi-threading, enabling the simulation to step through all current worlds concurrently.

To build the real-time, lightweight visualisation frontend decoupled from the high-performance simulation, TypeScript [37] and p5.js [38] were chosen. TypeScript [37] is chosen for its strict typing to enforce the data types communicated through the WebSocket, ensuring consistency between the frontend and the simulation. P5.js [38] is used to render the visualisation, chosen for its simplicity. For the bridge between the decoupled systems, `Tokio` [39] was chosen to spawn an asynchronous background thread, allowing the main environment to execute completely independently. The warp [40] web server framework was selected to manage the WebSocket connection, ensuring reliable, low-latency streaming of state data from the simulation to the frontend.

Finally, GitHub [41] was used for version control and pyproject.toml for seamless Rust-Python cross-compilation.

2.7 Starting Point

Before this project, I had extensive Python experience, foundational knowledge of neural networks, and recent industry exposure to Rust and PyO3. However, I had no prior experience with PyTorch, Variational Autoencoders, or deep Reinforcement Learning paradigms (such as PPO or MAPPO). Consequently, a significant initial phase was dedicated to learning these.

2.8 Preparation Summary

This chapter specified the cooperative environment as a Dec-POMDP, reviewed candidate MARL algorithms and selected MAPPO for this project. It also introduced the success criteria C1-5 and a MoSCoW analysis table, which, together, define what it would mean for the project to convincingly answer the research question.

Chapter 3

Implementation

3.1 Implementation Overview

This chapter describes the implementation of the performant MARL framework and communication protocols used in this dissertation. Specifically, it details the Rust-based vectorised environment, the MAPPO training pipeline, the communication protocols, and the supporting engineering (testing, logging and checkpointing). Together, these components satisfy the success criteria C1-C3 and provide the experimental platform evaluated in Chapter 4.

3.1.1 System Architecture

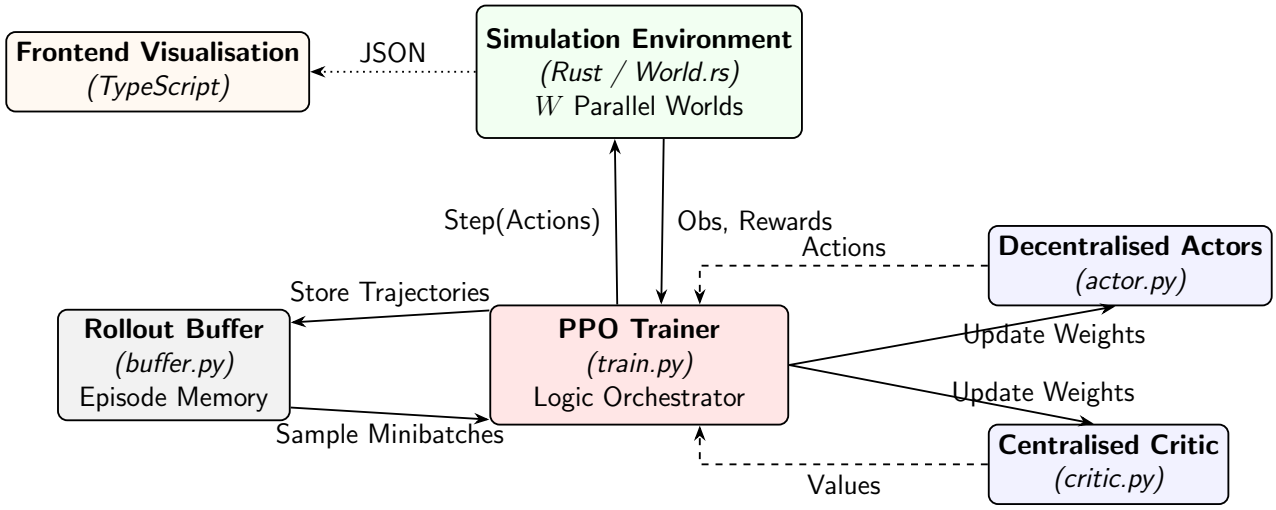


Figure 3.1: System Architecture Diagram: The PPO Trainer orchestrates the rollout phase with the Rust environment and the optimisation phase using data sampled from the Rollout Buffer.

The high-level system architecture (Figure 3.1) is divided into three sections: the Rust simulation backend, the Python MARL controller and the frontend visualisation. The simulation controls the state of all the current environments, and advances each environment at every step. The MARL controller updates and uses the policies, manages storage over the rollout phase and handles communication between agents in the same environment. The visualisation frontend receives data from the simulation via WebSocket for debugging and demonstration purposes.

3.1.2 Repository Overview

```
/ ..................................................................(13,722 LOC)
├── configs/ ..............................Global configuration files (135 LOC)
├── controller/ ......Python/PyTorch MARL logic and autoencoders (4,451 LOC)
│   ├── marl/comms/ ....................................Communication protocols
│   ├── marl/core/ ............Infrastructure (RolloutBuffer, Logging, Config)
│   ├── marl/models/ ............Neural networks (Actor, Critic, VAE Encoders)
│   └── marl/runners/ ............Training loops (PPO, Imitation, Autoencoder)
├── environment-rs/ .....High-performance vectorised environment (3,199 LOC)
│   └── src/ ............................Core logic (World, Agent, GridMap, Sim)
├── frontend/ ............................Real-time visualisation UI (268 LOC)
├── notebooks/ ..................Jupyter notebooks for evaluation (4,865 LOC)
└── tests/ ..............................Unit tests for controller/ (780 LOC)
```

The project is released open-source under the GPL-3.0 license to encourage open-source reproducibility and further research.

3.1.3 Engineering Practices

For the Rust-based environment, unit testing is rigorously applied. Using the `cargo` tool `llvm-cov`, a 97% coverage of functions is achieved by the test suite. In addition to unit testing, a lightweight real-time visualisation is engineered to provide immediate feedback during execution, making system behaviour easier to inspect as the environment runs.

In the Python controller, the core folder containing the infrastructure is also unit tested. Using the `pytest` library, 86% coverage of lines of code is achieved by the test suite. For the rest of the controller, unit tests were deemed inappropriate because they cannot evaluate macro-level MARL policy convergence.

To have an impact beyond the tripos, a README.md is produced to explain how to use the program. The README.md explains how to edit configuration files, add new communication protocols and evaluate the results (Appendix I).

3.2 Environment

The environment is designed specifically to satisfy C1 (Section 2.5.1) by providing a configurable, high-throughput environment.

3.2.1 Simulation Architecture

The core environment involved N agents whose job is to collect assigned ingredients from fixed stations or other moving agents. After collection, agents combine ingredients and deliver the output to a drop-off station within the 2D grid.

The environment is built as a two-dimensional grid of cells. At each timestep, agents can move to a neighbouring cell or interact with another entity in the same cell.

To support future protocol testing, the environment is highly configurable via a single file (Appendix H).

To ensure cooperation, agents cannot see the entities from which they must collect ingredients.

Reinforcement learning requires millions of datapoints [15] to produce a sufficiently usable policy. To collect this data efficiently, a vectorised environment with efficient data movement between the RL script and the Rust simulation is designed.

3.2.2 Cross-Language Interoperability

To allow the Rust simulation to be callable from the Python RL training scripts, `PyO3` [42] bindings are developed. A fundamental bottleneck of Python<=3.13 is the GIL [35]. The Rust library `PyO3` generates C-compatible Foreign Function Interface (FFI) bindings that can be called from the Python scripts. This FFI is compiled using setuptools-rust, a plugin for the setuptools library [43], that allows for the compilation of Rust extensions. This toolchain is defined within the `pyproject.toml` configuration enabling all building into a single installation command.

For interacting with the simulation from Python, a `SimWrapper` class is developed following the Adapter Design Pattern [44]. This wrapper isolates the reinforcement learning algorithm from the foreign function interface mechanics. PyTorch works in high-dimensional matrices; it is computationally faster to pass flat arrays to the Rust simulation to exploit CPU cache locality and throughput. To take a simulation step, the `SimWrapper` class reshapes the data and passes it to the Rust simulation. On return from the Rust simulation, `PyO3` provides a numpy [30] array, allowing zero-copy reshaping back to PyTorch's expected tensor shape in $O(1)$ time.

3.2.3 World Vectorisation

Due to Python's GIL, true parallelisation of CPU tasks is not efficient (see ablation study in Section 4.2.1). To circumvent this, we offloaded the parallelisation logic to the Rust simulation and stepped through multiple independent instances of the world concurrently. A single `Simulation` class is created to hold a vector of `World` states currently being simulated. According to Amdahl's Law [45], the theoretical speedup of a program is bottlenecked by its sequential fraction. The Rust library `Rayon` [36] is used to parallelise the computation of each world, thereby minimising the sequential overhead. This computation includes:

- Applying actions chosen from the policy
- Collecting rewards for feedback on the policy
- Collecting each agent's observations and each world's global observations for input to the actor and critic networks

The number of parallel worlds is determined through analysis (see SPS analysis in Section 4.2.1). However, machine computation differs, so the number of worlds in parallel can be modified in the configuration files. This vectorisation directly aids C1 by enabling hundreds of worlds to be simulated in parallel.

3.2.4 Spatial Hashing

To further address C1, an efficient spatial hashing method is used to optimise the per-step computation. In the environment, the closest entity (agent or station) to each agent is part of each agent's observations. A `GridMap` struct using spatial hashing replaces a naive $O(N^2)$ Euclidean distance calculation between N entities. Every world is represented as a `Vec<Vec<GridCell>>`, where agents register their presence upon movement, and stations register upon initialisation. Using this data structure, we can convert our $O(N^2)$ time complexity to $O(1)$. In addition to this functionality, we can easily calculate an exploration reward. Each `GridCell` contains an `is_explored` flag. When agents step on cells with this flag set to `false`, negate this flag and collect a small positive reward (Section 3.3.1 for further details).

3.2.5 Visualising the Environment

To help debug the environment, a lightweight, real-time visualisation is built in line with C1. Integrating the visualisation directly into the training loops would lead to strong performance degradation as each RL step would have to wait for a complete render of the current state.

To decouple these systems, an independent frontend is developed. On initialisation, the Rust backend spawns a background thread using the `Tokio` library [39] to manage the WebSocket without slowing down the main simulation, utilising the `warp` framework [40] to establish the connection and stream state data to the frontend. During runtime, the current state data is streamed from this WebSocket. Strict types are defined in both the Rust simulation and the frontend for the data communicated over the WebSocket, ensuring the system accurately reflects the current state.

To synchronise the configuration between the visualisation and the simulation backend, a Python utility extracts required parameters from the configuration files and places them into a frontend-friendly format. This synchronisation is integrated into the Vite [46] build script to ensure perfect synchronisation.

3.3 Multi-Agent Reinforcement Learning Infrastructure

The MARL infrastructure implements C2 (Section 2.5.1) using Multi-Agent Proximal Policy Optimisation, which includes a shared actor and centralised critic network.

3.3.1 Reward Function

Liu suggests that for these more complex games, a dense reward system should be implemented, allowing agents to gain useful feedback on every step.

Each agent i at timestep t receives a reward of r_t^i:

$$r_t^i = r_{\text{step}} + r_{\text{invalid}}(s_t^i, a_t^i) + r_{\text{interact}}(s_t^i, a_t^i) + r_{\text{explore}}(s_{t+1}^i) \tag{3.1}$$

Where:

- **Step Penalty (r_{step})** Agents receive a penalty of -0.005 every step to prevent idle behaviour.
- **Invalid Move Penalty ($r_{\text{invalid}}(s_t^i, a_t^i)$)** To teach the agents to stay within the confines of the grid, the agents receive:

$$r_{\text{invalid}}(s_t^i, a_t^i) = \begin{cases} -0.02, & \text{if } a_t^i \text{ is invalid} \\ 0, & \text{otherwise} \end{cases} \tag{3.2}$$

- **Interaction Reward ($r_{\text{interact}}(s_t^i, a_t^i)$)** Interaction rewards are awarded when an agent executes the interact action on a visible station or another agent. The reward consists of a discovery term and a task-progress term:

$$r_{\text{interact}}(s_t^i, a_t^i) = \begin{cases} r_{\text{found}} + r_{\text{progress}}, & \text{if } a_t^i = \texttt{Interact} \\ 0, & \text{otherwise} \end{cases} \tag{3.3}$$

where

$$r_{\text{found}} = \begin{cases} 0.5, & \text{if the interaction reveals a previously unknown relevant station or agent} \\ 0, & \text{otherwise} \end{cases} \tag{3.4}$$

and

$$r_{\text{progress}} = \begin{cases} 1.0, & \text{if the interaction completes a recipe combination} \\ 0.7, & \text{if the interaction is successful but does not complete a combination} \\ 0, & \text{if the interaction has no effect} \end{cases} \tag{3.5}$$

- **Exploration Reward ($r_{\text{explore}}(s_{t+1}^i)$)** If an agent enters a previously unvisited square, a small reward is added to encourage exploration.

$$r_{\text{explore}}(s_{t+1}^i) = \begin{cases} 0.002, & \text{if } s_{t+1}^i \text{ is the first to visit the cell} \\ 0, & \text{otherwise} \end{cases} \tag{3.6}$$

3.3.2 Actor and Critic Design

Similar to Liu [6], an actor-critic network is set up following the Centralised Training Decentralised Execution (CTDE) paradigm. The PyTorch module `PPO_Critic` operates solely during the training phase and receives global knowledge of the current state of the world. The global knowledge is the complete, unmasked environmental state. This is processed by a deep Multi-Layer Perceptron (MLP). To maintain network stability, `nn.LayerNorm` and ReLU activation functions are applied sequentially. The network outputs an estimate of each agent's return for the given state.

The PyTorch module `PPO_Actor` controls each agent and acts only using local knowledge of the agent. This local knowledge is the agent's own state (position, inventory - what the agent currently has and needs), the closest entity if within a specific radius and the communication vectors broadcast by all other agents. A deep MLP processes this, then passes it through a

Long Short-Term Memory (LSTM) unit. The LSTM allows individual agents to remember the locations of stations and meeting points with other agents. The network outputs action logits (probabilities of each action) and returns the LSTM output for the communication protocol to use later.

To prevent weights from vanishing or exploding during early training, orthogonal initialisation is used instead of PyTorch's default uniform initialisation.

3.3.3 Memory Management

During the rollout phase, the data collected, including inputs to the policy (communication values, observations) and the feedback from the simulation (rewards), must be saved for the update phase. In the vectorised environment with multiple agents over hundreds of timesteps, simply appending to a Python list is insufficient due to constant copying overhead when the list doubles in size [47] and memory fragmentation [48] (Section 4.2.2 for ablation study). To prevent this overhead, a custom `RolloutBuffer` class is developed. On instantiation, the buffer preallocates contiguous `torch.tensor`s on the target device for rollout data: local/global observations, LSTM hidden states, per-agent targets/rewards, broadcast communication vectors with log-probabilities, actions with log-probabilities, critic values, and GAE-derived advantages/returns.

After each environment step, data is directly inserted into the pre-allocated tensors via index slicing. This insertion is a $O(1)$ zero-copy operation. This method minimises the data transfer across the CPU-GPU boundary, which is an expensive operation.

3.3.4 Generalised Advantage Estimation

After concluding the rollout phase, the temporal difference returns and the advantages need to be calculated to update the policies. To avoid unnecessary data movement, this process is executed directly on the stored tensors inside the `RolloutBuffer` instance. As detailed in Algorithm 2 (see Appendix B), GAE recursively accumulates the one-step Temporal Difference Error using the decay parameter (λ) and discount factor (γ) to balance the bias-variance trade-off. By adding the current δ to the exponentially discounted future rewards, GAE effectively balances the bias-variance trade-off inherent in deep reinforcement learning. The target returns for training the Critic are formulated by summing the advantage with the current state value. This algorithm has a time complexity of $O(T)$.

3.3.5 Communication Strategy Pattern

To avoid repetition of code blocks and consistency over each protocol, an abstract base class is engineered (`Comms`) using the Python library `abc` to require each subclass to implement the necessary methods. Each protocol is implemented as a PyTorch module that inherits from the aforementioned base class and `torch.nn.Module`. On initialisation, the `actor` module dynamically instantiates the required protocol based on the configuration. This enabled seamless experimental hot-swapping of protocols without altering the MAPPO algorithm.

3.3.6 Multi-Agent Proximal Policy Optimisation Algorithm

Algorithm 1 (see Appendix A) depicts the update loop for MAPPO. To ensure stability in the updates, advantage normalisation is applied to the GAE advantages [49].

For every epoch, the data collected by the rollout phase is split into random minibatches. First, the critic is evaluated by feeding the global observation into the GAE returns using Mean Squared Error (MSE) loss.

The communication codes broadcast by all other agents (excluding the agent itself) in the preceding time step are aggregated and fed into the actor network along with the local observations and the LSTM hidden state. To stabilise policy updates, the standard PPO clipped surrogate objective is used. The communication and action policies are optimised using separate clipped objectives, each with its own probability ratio. This strict isolation ensures that a confident, but incorrect, communication policy update does not destructively skew the gradients of a well-performing spatial movement policy.

Each of these surrogate losses is aggregated into a single composite loss, along with an entropy loss to encourage continued exploration. However, before applying the final gradients to the networks, the complete actor and critic parameters are clamped to a maximum L_2 norm of 0.5.

Due to imitation learning (as detailed in Section 3.6.3), the actor and critic are partially pre-trained. The actor uses separate learning rates to preserve this pre-trained behaviour: core policy components (e.g., LSTM and action heads) use a higher rate, whilst communication parameters use a lower rate via `PyTorch`'s parameter groups. The critic uses a standard single learning rate to remain responsive to policy changes. This careful orchestration of advantage normalisation, isolated loss clipping, gradient clamping and variable learning rates ensures monotonic policy improvement across the training landscape.

3.4 Communication Protocols

Communication allows the agent to mitigate the POMDP effect, improving the mean task reward per step. Every agent broadcasts a code to all other agents, which is received in the next simulation step. Agents do not receive their own code.

Making the software reusable and flexible by allowing communication protocols to be interchangeable will enable comparisons between protocols and help satisfy C5 (Section 2.5.1).

- **No Communication**

  The No Communication baseline lacks a communication head and ignores external messages.

- **On-Policy Fixed Discrete Communication**

  Another comparable baseline is required, one with communication. For this, each agent broadcasts a set of discrete values learned jointly with its policy. To make the discrete value differentiable, the Straight-Through Estimator (STE) is used (explained in Section 3.5.2). This yields a simple, differentiable communication.

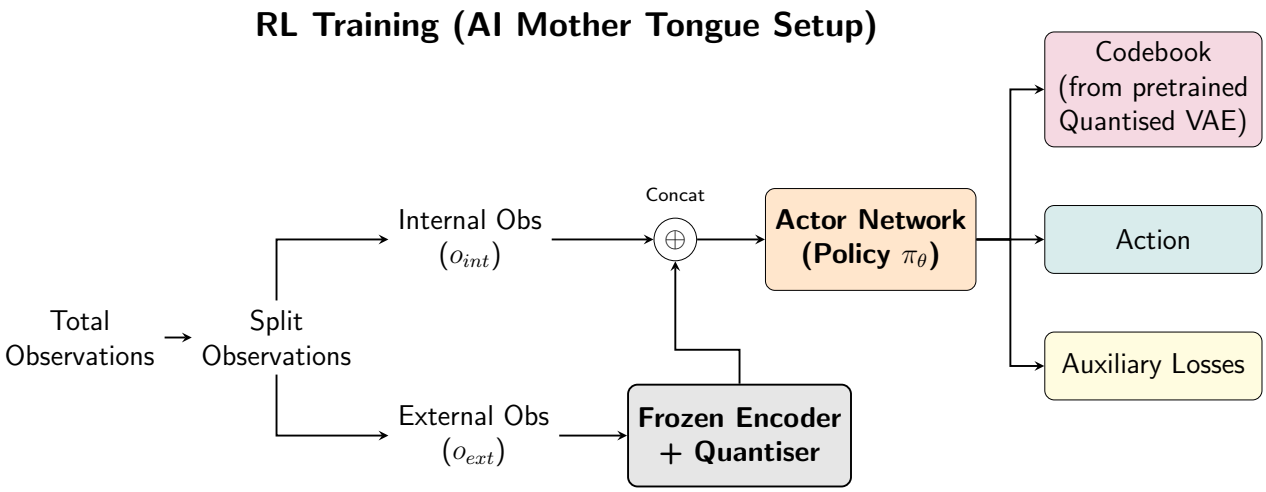


Figure 3.2: Architecture diagram for frozen VAE pretraining and RL training the actor network.

- **AI Mother Tongue**

  AI Mother Tongue communication protocol extends Liu's proposed architecture [6] for this more complex environment. Figure 3.2 shows my extension to the original AI Mother Tongue (Figure 2.2). The agents' observations are split. The external observations (those which represent the agent's view of the environment) are split from the rest and fed through the frozen encoder and quantiser. Another key difference is the action head. Instead of the simple rule Liu implemented, we pass the policy output through an MLP to output a probability distribution over the action space.

- **WhisperWise: On-Policy Discrete Communication with Reflection Strategies**

  To isolate the effect of the reflection strategies independent of the frozen VAE encoder, a novel ablation architecture is engineered. This protocol adds the reflection strategies detailed in Section 3.7 to the on-policy discrete communication protocol as described in Section 3.4

3.5 Variational Autoencoders

The VQ-VAE and SQ-VAE are implemented to support C4 (Section 2.5.1), allowing rich codebook utilisation and strong semantic meaning of each token.

3.5.1 Autoencoder Architecture

One objective of this project is to investigate different Variational Autoencoders to develop this AI Mother Tongue Language. Vector-Quantised Variational Autoencoder (VQ-VAEs) and Stochastically-Quantised Variational Autoencoder (SQ-VAEs) are the two autoencoders that will be evaluated. To allow for experimentation, interchangeability is key. An agnostic `Encoder` class is engineered for the shared neural network boilerplate, latent dimension mapping, and disk I/O. This allows the VQ-VAE and SQ-VAE to be designed as lightweight, interchangeable models, satisfying our goal.

The `Encoder` class defines the reconstruction loss for all VAE variants. The observation space x is split into continuous features x^{cont} and binary features x^{bin}.

$$\mathcal{L}_{\text{recon}} = \|x^{\text{cont}} - \hat{x}^{\text{cont}}\|_2^2 + \lambda_{\text{bce}}\,\text{BCE}\big(x^{\text{bin}}, \hat{x}^{\text{bin}}\big), \tag{3.7}$$

where λ_{bce} balances the scale of the Binary Cross Entropy loss (BCE) term relative to the MSE term.

3.5.2 Vector-Quantised Variational Autoencoder

Following van den Oord et al. [26] and Liu [6], a Vector-Quantised Variational Autoencoder (VQ-VAE) is implemented. The standard structure is utilised: Encoder, Quantiser and Decoder.

A challenge with VQ-VAEs is that the argmin operator is non-differentiable. Since the mapping is a step function, the true Jacobian matrix $J_{true} = \frac{\partial z_q}{\partial z_e}$ is zero almost everywhere and is undefined at the boundaries, preventing backpropagation. To prevent this, the straight through estimator (STE) [50, 51] approximates the Jacobian as the identity matrix I during the backward pass.

$$J_{approx} = \frac{\partial z_q}{\partial z_e} \approx I \tag{3.8}$$

This allows gradients to flow from the decoder to the encoder. However, as this is an approximation, there exists a gradient bias error $E = \nabla_{z_e}\mathcal{L}_{STE} - \nabla_{z_e}\mathcal{L}_{true}$. If this error were to grow, the PPO surrogate objective could force the policy ratio $r_t(\theta)$ to consistently hit the clipping bounds, halting learning. The addition of the commitment loss term ($\mathcal{L}_{\text{commit}}$) ensures the bias E remains small by strictly penalising the distance between the continuous vector and the closest codebook entry, preserving the stability of PPO updates.

3.5.3 Stochastically-Quantised Variational Autoencoders

A known issue of using learned codebooks is codebook collapse. To fix this issue, Takida et al. [27] propose Stochastically-Quantised Variational Autoencoder (SQ-VAE), a variational Bayes framework for learning VQ-VAE components.

Instead of using the argmin operator, we compute a categorical probability distribution over the codebook by assigning each code a probability proportional to its distance from the encoder output. Early during training, the probability distribution is wide, which helps explore the entire codebook. Later on during training, the probability distribution shrinks, forcing the model to make hard choices.

A temperature annealing scheduler is implemented to shrink the probability distribution over training. To control this sharpening behaviour, a temperature annealing scheduler is applied to the categorical distribution.

$$\tau = \max(\tau_{\min}, \tau_{\text{old}} \cdot \text{annealing\_rate}) \tag{3.9}$$

However, this method still uses Euclidean distance, whereas in tasks such as NLP and audio, the semantic meaning of a feature is often entirely captured by the vector's direction, not its magnitude [52]. To improve this method for our use case, we use an extension of this model: the von Mises-Fisher Stochastically-Quantised Variational Autoencoder (vMF SQ-VAE).

To make this differentiable, we use the Gumbel-Softmax trick [53, 54]. In the forward pass, we force the output to be a strict one-hot vector. In the backward pass, we act as if the output were continuous, following this formula: vMF SQ-VAE uses cosine similarity instead of Euclidean distance.

This is done by first normalising the variables onto a unit sphere.

$$z_{norm} = \frac{z_e}{\|z_e\|_2}, \quad e_{norm,k} = \frac{e_k}{\|e_k\|_2} \tag{3.10}$$

The code distribution is then defined via a von Mises-Fisher-like softmax over cosine similarities:

$$q(z_q = e_k \mid z_e) = \frac{\exp\left(\kappa\, z_{\text{norm}}^{\top} e_{\text{norm},k}\right)}{\sum_j \exp\left(\kappa\, z_{\text{norm}}^{\top} e_{\text{norm},j}\right)}, \tag{3.11}$$

where κ controls how concentrated the distribution is around the most similar codes.

Equation 3.11 is computed with PyTorch's `torch.nn.functional.gumbel_softmax` function. The temperature used in the annealing scheduler is applied to the Gumbel Softmax temperature argument.

The loss function relies solely on L_{recon} as explained in Equation 3.7.

3.6 Offline Pre-training

3.6.1 Expert Agent and Data Collection

Data for offline pre-training is collected from an expert agent. In the game, creating an optimal agent is intractable because we must decide the optimal route to multiple stations. This expert agent views the unmasked, global observation state. The expert agent then employs a greedy heuristic, taking steps directly to the nearest required station until interaction is possible. By deploying this greedy expert agent, a high-quality dataset of spatial movements can be collected.

3.6.2 Training VAEs

The AI Mother Tongue framework requires these autoencoders to be trained before the RL phase of the learning process.

A rich codebook is vital for adequately describing external observations. A significant challenge in training autoencoders is codebook collapse, where only a small subset of the codebook's embeddings is used. To mitigate this, the codebooks are not randomly initialised; instead, data-driven codebook initialisation is used to set the codebook's spatial priors.

Algorithm 3 (see Appendix C) shows how the priors are collected and set. A subset of the data collected from the expert agent is used to estimate the spatial priors by passing it through the initialised, but untrained, encoder. Centroids are then found by using the `MiniBatchKMeans` function from the Python library `scikit-learn` for K clusters (where K is the vocabulary size). If the current VAE is of type vMF SQ-VAE, then we must also normalise the embeddings onto a unit sphere with `torch.nn.functional.normalize(..., p=2.0)`. These centroids overwrite the randomly initialised embeddings in the codebook.

After learning the VAE, it is loaded into the encoder for the RL phase, and the parameters are frozen to prevent further training. This is done in PyTorch via `parameter.requires_grad = False`.

3.6.3 Imitation Learning

Training the RL phase takes a significant amount of time. To reduce the training time required, Liu suggests training both the actor and critic models with supervised learning directly before the RL phase [6]. Required observations, optimal actions and their associated advantages can be pre-collected from an expert agent.

Using this data, supervised learning is applied to the actor and critic networks. To simulate communication, we send the agent's current nearest target location in its communication slot, encoded via a learned codebook lookup from spatial binning.

The world is broken into K discrete areas. K is chosen such that the world is discretised into no more spatial bins than there are codes in the vocabulary, to balance two effects: avoiding unrealistically precise target locations that the communication channel cannot represent, whilst still providing sufficiently precise spatial information. Each area is assigned a unique ID, unique_id $= y \times$ world_width $+ x$, where x and y are the discrete column and row indices. A lookup in the codebook using `unique_id` is done, and the result is injected into the communication channels during training.

The actor outputs action logits, and cross-entropy is calculated for the loss function:

$$\mathcal{L}_{\text{actor}} = -\log \pi(a_b^* | s_b, c_b) \tag{3.12}$$

The critic predicts the returns and uses MSE to calculate the loss function:

$$\mathcal{L}_{\text{critic}} = \mathbb{E}_t \left[(V_\phi(S_t) - R_t)^2 \right] \tag{3.13}$$

On completion, these models are saved to disk and loaded when the reinforcement learning begins.

3.7 Reflection Strategies

A major emphasis in Liu's work is the importance of reflection strategies [6]. They introduce additional loss terms during the RL phase to foster more complex learning and communication beyond simple reward maximisation. These reflection losses are defined over *sender* and *receiver* contexts constructed from observations and communication symbols. Due to the complexity of the environment, these auxiliary losses were adjusted to be more task-specific.

The two different inputs to the auxiliary heads are defined as:

1. *sender_context* (c_i^{send}) defined as the concatenation of the output of the actor's recurrent unit and the communication vector that the actor broadcast.

2. *receiver_context* (c_i^{recv}) defined as the concatenation of the output of the actor's recurrent unit and all other agents' communication vectors.

1. **Predicted Return Loss**

   An auxiliary head is trained to predict the true return R_t collected from the GAE advantage estimation. Loss is defined by:

$$\mathcal{L}_{\text{return}} = \frac{1}{N} \sum_{i=1}^{N} \left(\hat{R}(c_i^{\text{send}}) - R_i \right)^2, \tag{3.14}$$

   This mirrors Liu's value-prediction reflection, aligning communication with future rewards.

2. **Intent Loss**

   The intent loss described by Liu models how agents' messages influence partners' behaviour and rewards; however, this is not suitable for our needs. Instead, the agents' own target is predicted using **sender context** and other agents' targets are predicted using **receiver context** (explicitly promoting a Theory-of-Mind style understanding of partners' intentions [55, 56]).

   These losses are defined as.

$$\mathcal{L}_{\text{sender-goal}} = \frac{1}{N} \sum_{i=1}^{N} \eta_i \left\| \hat{\tau}^{\text{send}}(c_i^{\text{send}}) - \tau_i \right\|_2^2, \tag{3.15}$$

$$\mathcal{L}_{\text{receiver-goal}} = \frac{1}{N} \sum_{i=1}^{N} \sum_{\substack{j=1 \\ j \neq i}}^{N} \eta_j \left\| \hat{\tau}^{\text{recv}}(c_i^{\text{recv}}, j) - \tau_j \right\|_2^2, \tag{3.16}$$

   where

   - $\hat{\tau}^{\text{send}}$ predicts the sender's own target from its context
   - $\hat{\tau}^{\text{recv}}$ predicts another agent's target from the receiver context
   - τ_i is the target of agent i
   -

$$\eta_i = \begin{cases} 1, & \text{if agent } i \text{ has a valid target,} \\ 0, & \text{otherwise.} \end{cases}$$

   The total intent loss is then:

$$\mathcal{L}_{\text{intent}} = \mathcal{L}_{\text{sender-goal}} + \mathcal{L}_{\text{receiver-goal}}. \tag{3.17}$$

Thus, the total reflection loss is:

$$\mathcal{L}_{\text{refl}} = \lambda \left(\mathcal{L}_{\text{return}} + \mathcal{L}_{\text{intent}} \right), \tag{3.18}$$

with a small coefficient λ (e.g. 0.02) so that reflection regularises, rather than dominates, the RL objective.

This loss is returned and added to the composite MAPPO objective function for backpropagation.

3.8 Software Engineering

MARL systems take thousands of rollout phases to train. Leaving the system for evaluation until the end of training would render debugging practically impossible. To ensure strict scientific rigour, custom observability and checkpointing frameworks were implemented supporting C5 (Section 2.5.1).

- **Real-time System Monitoring**: A lightweight `MetricTracker` collects program statistics which are continuously flushed to disk via a `Logging` class to prevent data loss in case of hardware failure. Additionally, a `Visualiser` class was developed using the Python library `rich` [57] to display relevant, real-time statistics for that run.
- **Configuration Management**: A modular configuration system is engineered to manage the large number of hyperparameters, with separate configuration files for actor and critic hyperparameters, imitation learning, VAE training, reinforcement learning, environment settings, communication protocols, and MAPPO. The configuration files allow seeds to be set for easy reproducibility.

  To prevent silent errors, the Python library `pydantic` is used. The configuration dataclass is inherited from `pydantic`. `BaseModel` defines the expected schema for each of the aforementioned files. This data class is then passed throughout the program.
- **Checkpointing**: Simply storing the logs accumulated during the run is insufficient for resuming training in the event of hardware failure. For this functionality, a `CheckpointManager` class is developed. On instantiation, a config file is saved to a new folder for future reference. Then, every X training steps (e.g., $X = 100$, configurable in the configuration files), the actor and critic models, along with their respective optimisers, are saved. These saved models and optimisers can be loaded on startup to reproduce any results.

3.9 Implementation Summary

This project makes several contributions to the field beyond the scope of a dissertation. The project provides a framework for communication in MARL, which includes a configurable, vectorised, high-throughput MARL simulation and a MAPPO-based controller with interchangeable communication protocols. This allows other researchers to plug in new alternative protocols and VAE bottlenecks and compare them consistently with others.

The project also extends the AI Mother Tongue architecture [6] to a more complex cooperative environment and experimented with VAE variants. By using the custom framework with logging tools and checkpointing, training and the semantic meaning of messages can be analysed in detail, as shown in Chapter 4.

Chapter 4

Evaluation

This chapter evaluates the implemented framework and communication protocols against the success criteria defined in Section 2. It first measures system performance, then compares task reward and learning using different communication protocols, and finally analyses the structure and semantics of the emergent communication, including ablations and a more complex task. These results directly address whether AI Mother Tongue and the novel WhisperWise protocol enable effective emergent discrete communication.

4.1 Evaluation Setup

A specific cooperative simulation setup is evaluated. There exist two agents. Each agent has two ingredients to collect from unique stations. They also each have a station to drop off their product. Each agent cannot observe their own pick-up stations, but can observe their teammates at an infinite range. This setup ensures cooperation is required and, therefore, communication must emerge for optimal task reward.

The world is defined as a 5x5 grid. This is chosen to be small enough to minimise the spatial navigation problem, allowing the evaluation to focus on the emergent language. For the 5x5 grid, agents with no communication must take on expectation ~ 35 actions to find both stations (see Appendix F). Therefore, to reduce the difference in mean task reward per step between the communication and no-communication protocols, the number of actions agents are allowed to take in each world is limited to 30.

The use of standardised hyperparameters is preferred over aggressive grid searches to demonstrate the robustness of the emergent communication.

A batch size of 512, 57,600 datapoints per update phase (Section 4.2.1) and four optimisation epochs are selected to optimise stability and sample efficiency. The batch size of 512, a common power-of-two choice in machine learning, reduces gradient variance for LSTM-based actor-critic networks whilst maximising GPU utilisation. The low four epochs prevent overfitting to stale data during update phases, aligning with PPO recommendation of 3-10 epochs [7].

These communication hyperparameters apply to all communication protocols.

A vocabulary size of 32 is chosen as the optimal point on the elbow curve (Figure 4.1) where the model captures maximum environmental context whilst balancing a manageable search space for the agents. Each code in the codebook is of size eight to allow sufficient spatial encoding without introducing excessive computational complexity.

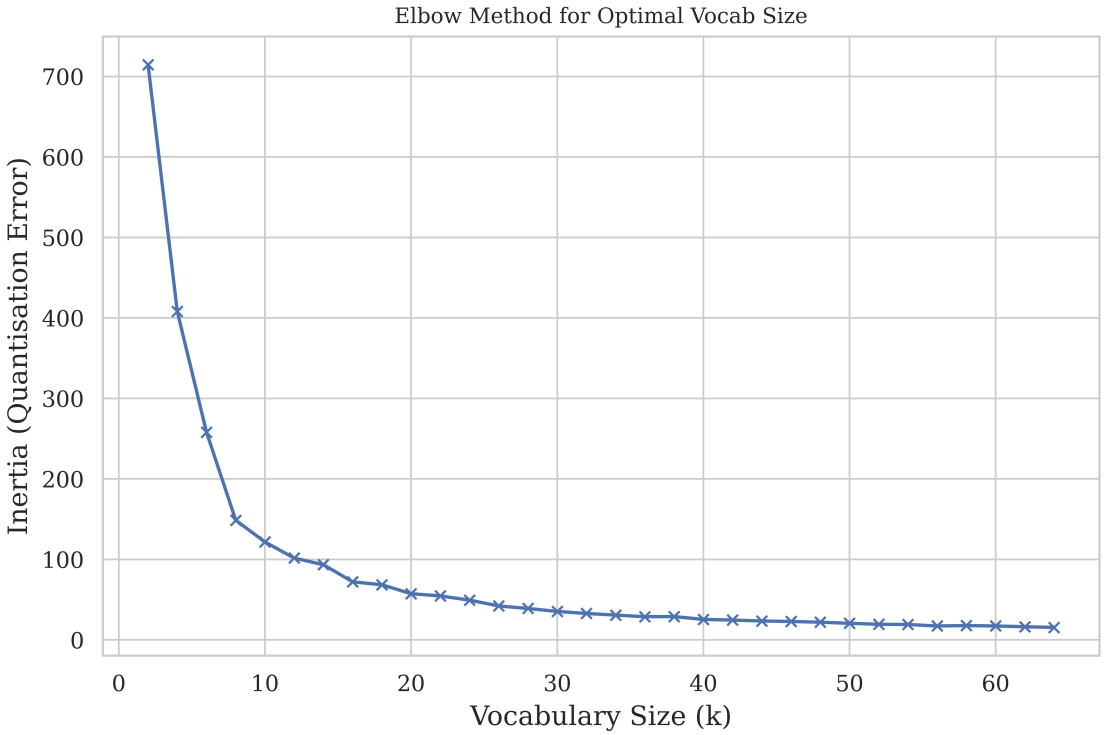


Figure 4.1: Illustrates the elbow curve used to determine vocabulary size.

4.2 System Performance

Reinforcement Learning with multiple agents requires millions of datapoints for policy updates and an equal number for evaluating the language and policy. This section quantitatively measures the performance gains through the custom systems developed. Specifically, it analyses the steps-per-second of the Rust simulation developed and the memory efficiency of the `RolloutBuffer`. Benchmarks were evaluated on NVIDIA RTX 5060 Ti and AMD Ryzen 7 3700X.

4.2.1 Vectorised Environment Throughput

A major issue with Python's concurrent threading is the GIL. While Python's built-in library `multiprocessing` can, in theory, bypass this limitation, it introduces considerable overhead. For this RL project, all threads require synchronisation at every simulation timestep.

To justify developing the simulation in Rust rather than Python, whilst evaluating C1, a simple mock simulation was developed in Python. This mock simulation was compared to the vectorised Rust backend. Environment steps per second (SPS) were calculated to analyse the throughput of each simulation. Number of worlds parallelised was scaled ($W \in 2^0, 2^1, ..., 2^{12}$) to measure concurrency efficiency.

As shown in Figure 4.2, the Python baseline has over 2x less SPS than the Rust simulation. Despite being able to use multiple CPU cores, the serialisation and the Inter-Process Communication overhead required to transfer the necessary data between isolated processes at every time step heavily bottleneck the throughput.

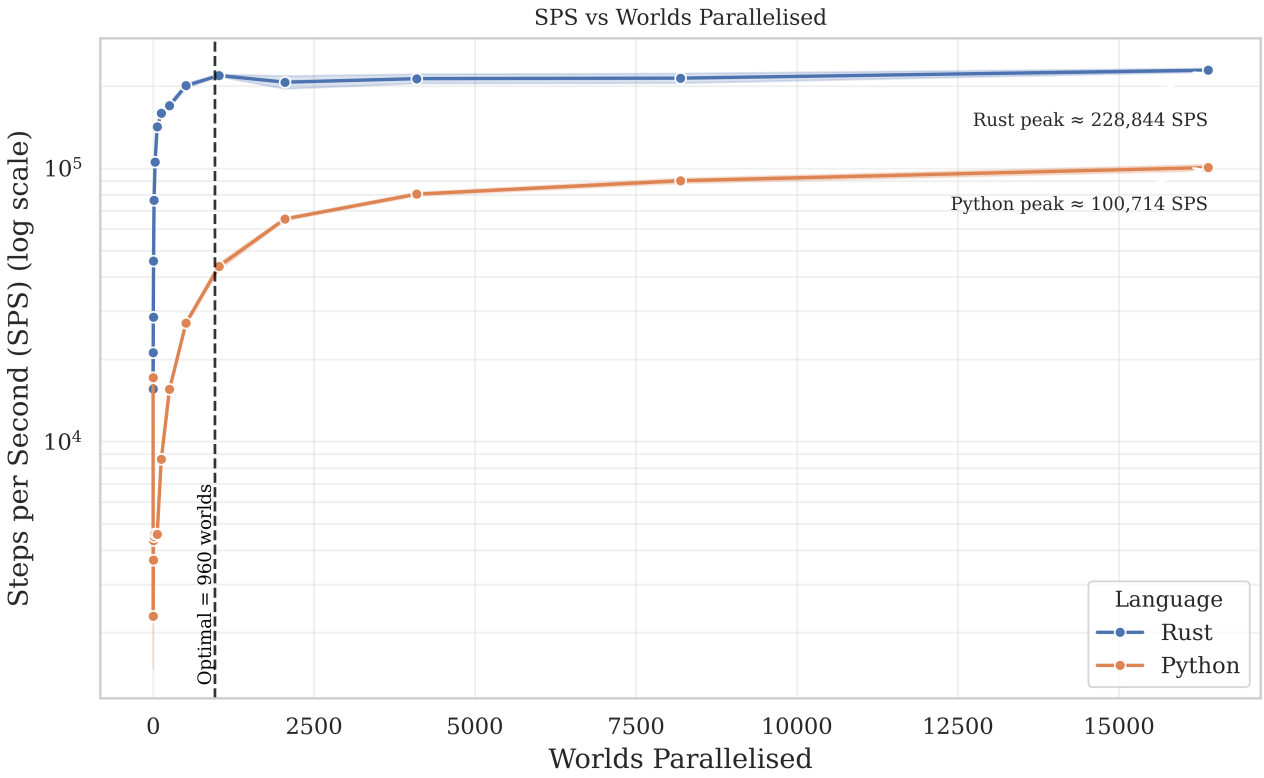


Figure 4.2: Concurrent Environment Throughput Performance. Comparison of Python multiprocessing and Rust with Rayon across an exponentially increasing number of parallel worlds. Shaded regions show one standard deviation across 10 runs.

In contrast, Rust can utilise `Rayon`'s thread-pool to manage the parallel world steps efficiently. Throughput scales with the number of worlds parallelised, reaching $228,844$ SPS. By using Rust's native `Rayon` threading, the sequential fraction of the program is minimised per Amdahl's Law [45], allowing near-perfect scaling that saturates the CPU cores.

For the evaluation, 960 worlds were parallelised across a single rollout phase. This maximised the parallelisation whilst also collecting an apt amount of datapoints for the update phase (960×30*(simulation steps per rollout)* $\times\ 2$*(number of agents)* $= 57,600$).

4.2.2 Rollout Buffer Efficiency

A secondary major potential bottleneck is transferring the results of the rollout phase to the training phase (across the CPU-GPU boundary). To evaluate the `RolloutBuffer`, the memory allocation and transfer overhead are compared to a baseline method (appending to Python lists and executing `torch.stack` after the rollout phase).

Memory Strategy	Allocation Time (s)	Peak VRAM Overhead (MB)
Dynamic Python Lists	6.47 ± 0.18	1213.45
Pre-allocated RolloutBuffer	6.79 ± 0.23	598.23

Table 4.1: Memory Management Overhead per Epoch: $T = 20,000$, $W = 32$.

As shown in 4.1, the `RolloutBuffer` method reduces VRAM by over 50%. This is because Python's dynamic arrays often require copying their contents from one memory location to

another to increase their size. Whilst there is a $\sim 5\%$ improvement in allocation time when using Python's dynamic list, the large improvement in VRAM will enable scaling this method to millions of datapoints.

4.3 Online MARL Task Performance

Before deploying the AI Mother Tongue architecture, the discrete bottlenecks were evaluated. To ensure optimal performance, an SQ-VAE with a data-driven prior was selected; the offline ablation studies justifying this architectural choice are detailed in Appendix D. To ascertain the effectiveness of the resulting AI Mother Tongue protocol, its task performance is evaluated against two baseline communication protocols.

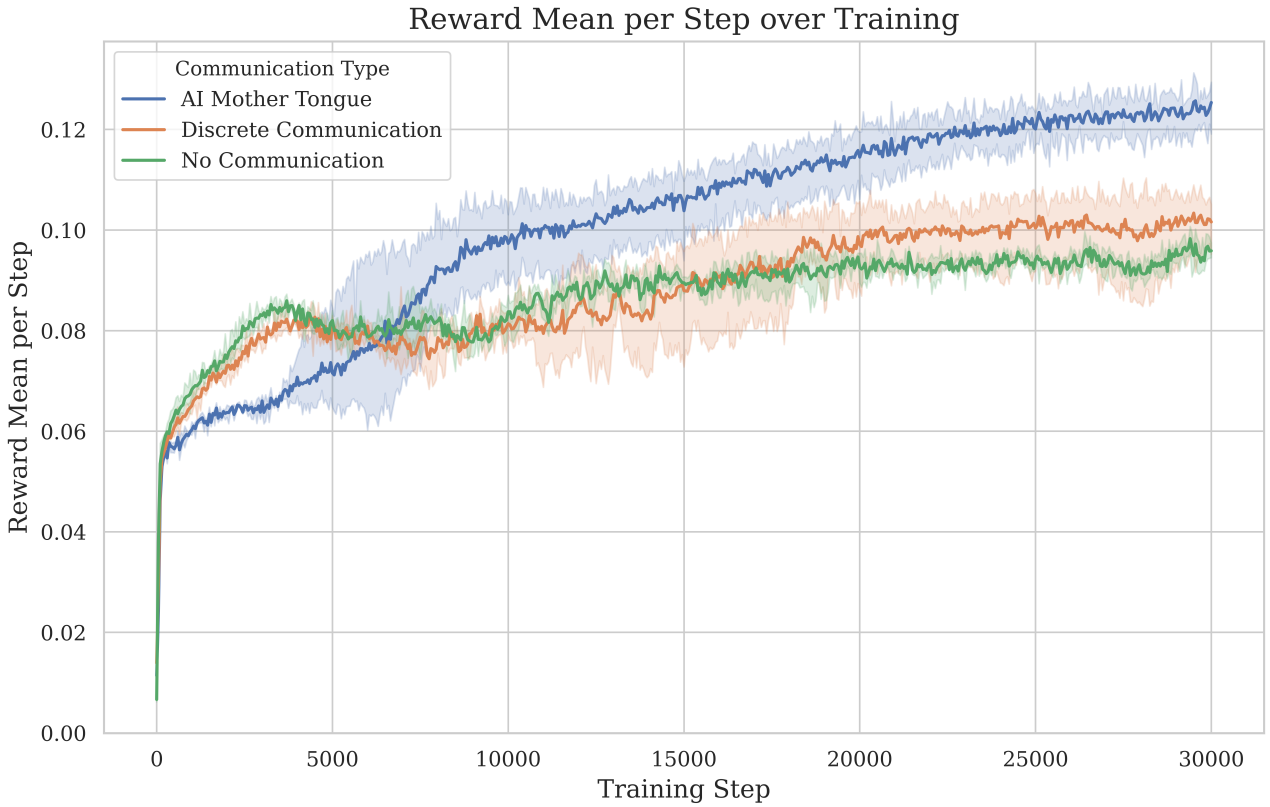


Figure 4.3: Reward mean per step over 30,000 PPO training steps. Each protocol was trained across three runs. A 95% confidence interval is shown.

To rigorously evaluate the performance differences, a one-way Analysis of Variance (ANOVA) is conducted on the mean task reward per step using 512 environment simulations. ANOVA assumptions were first checked: Shapiro-Wilk tests indicated approximate normality for all three protocols, and Levene's test indicated homogeneous variances. ANOVA found a significant effect of protocol performance, $F = 28.27, p < 0.001$. Tukey's post-hoc test showed AI Mother Tongue differed significantly from both Discrete Communication and No Communication, whilst Discrete Communication and No Communication did not differ significantly from each other. Based on mean scores, AI Mother Tongue achieved the best performance overall (matching training performance in Figure 4.3).

The expert agent achieves a mean reward per step of 0.24, highlighting the performance gap imposed by the agents' partial observability and confirming the difficulty of the POMDP

Training MARL policies often requires 10x or 100x as many steps as 30,000 steps. However, for this evaluation, 30,000 steps are sufficient, as the core research focus (spontaneous semantic compression) emerged within the first 12,000 steps (Section 4.4.1).

4.4 Information-Theoretic & Linguistic Properties

Sections 4.4 - 4.6 provide the deeper evaluation required by C4.

4.4.1 Spontaneous Semantic Compression

Throughout training, the agent's vocabulary perplexity is tracked. Communication perplexity measures the agent's uncertainty about the next token, defined as $e^{-\text{entropy of vocabulary}}$.

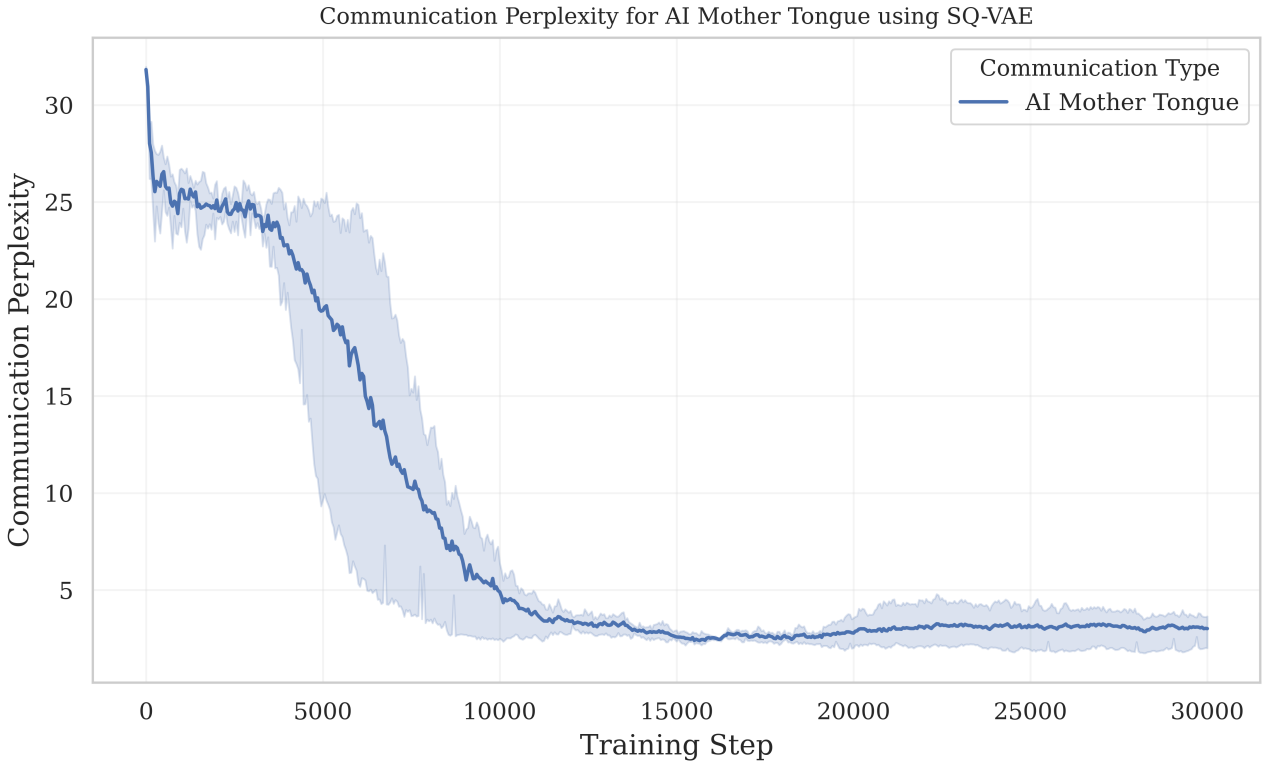


Figure 4.4: Communication Perplexity tracked over a training run of AI Mother Tongue when using SQ-VAE trained across three runs. A 95% confidence interval is shown.

At the start of training, the agents explore all possible outputs, leading to uniform random sampling across most of the codebook. From Figure 4.4, the RL begins to solve the asymmetric information task (between step 3,000 and 10,000), communication perplexity collapses and stabilises at ~ 3.0. This collapse represents spontaneous semantic compression. Instead of using the codebook's theoretical capacity, a small subset of codes is assigned meaningful semantic meaning.

4.4.2 Codebook Utilisation

To further understand the agent's use of language, the frequency of each code is analysed.

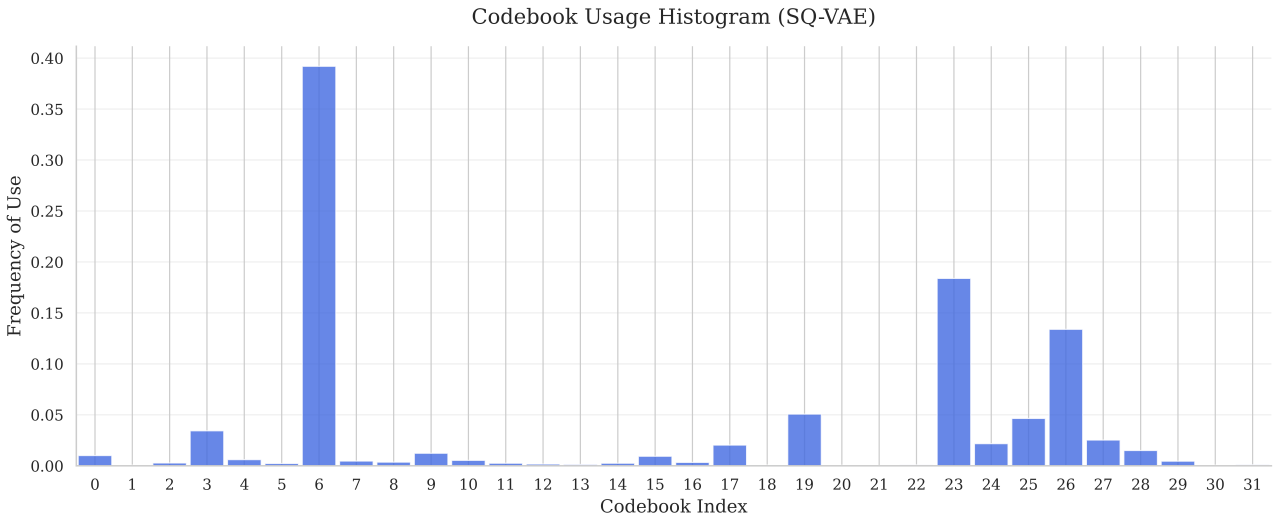


Figure 4.5: Normalised frequency of each code across 2880 runs.

From Figure 4.5, the distribution exhibits a strong skew, giving a low entropy score of 2.98 and a high Gini coefficient of 0.78. When visualising this empirical distribution with a rank-frequency plot (Figure 4.6), it demonstrates exponential decay resembling Zipf's Law [58], especially for the first 23 codes.

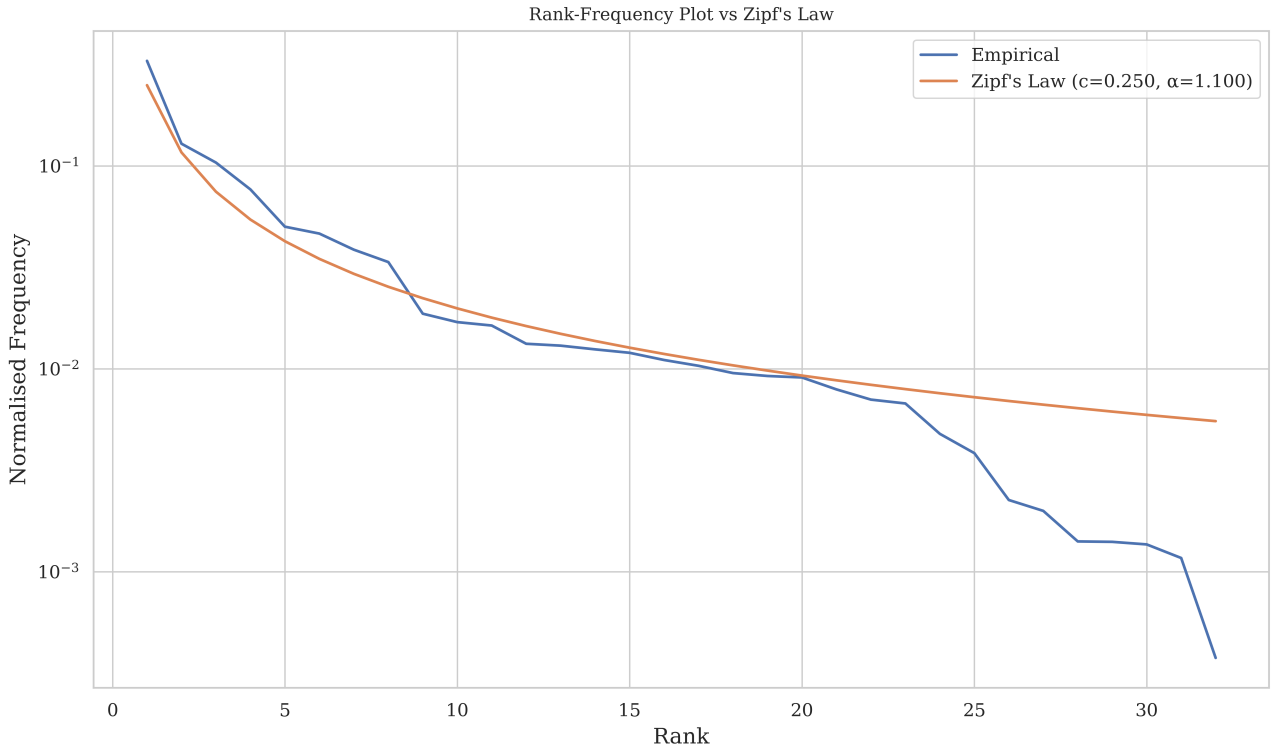


Figure 4.6: Vocabulary rank-frequency plot compared to Zipf's Law.

However, a notable deviation from Zipf's Law does occur at the tail-end of the distribution where the frequency rapidly decreases. Although a sudden drop to zero is not a feature of a classic Zipfian distribution, it is a feature of the truncated Zipfian distribution [59]. This occurs because, in most languages analysed, an infinite number of words can be created; however, in this system, there exists only a finite set of codes. The emergence of this phenomenon suggests that the agents were successful at assigning high-frequency tokens to common semantic meanings and low-frequency tokens to unnecessary ones.

4.5 Cognitive Mapping and Spatial Memory

To understand what the agent concretely knows, six strict linear probes were developed following Guillaume et al.'s methodology [60]. Memory is tested for allocentric (information stored using a global coordinate system) and egocentric (information stored relative to the self) memory. Each probe takes as input the final layer of the Long-Short Term Memory (LSTM) module, and predicts one of the following using MSE loss:

- The agent's own next target's coordinates (tests for allocentric memory)
- The agent's partner's next target's coordinates (tests for allocentric memory)
- The agent's own next target's relative coordinates from the agent's coordinates (tests for egocentric positional memory)
- The agent's partner's next target's relative coordinates from the agent's coordinates (tests for egocentric positional memory)
- A unit vector in the direction of the agent's next target (tests for egocentric directional memory)
- A unit vector in the direction of the agent's partner's next target (tests for egocentric directional memory)

Initially, in a simulation run, agents have no way of knowing their targets' locations. To account for this, the models are trained only on the last half of the timesteps to ensure the LSTM contains the necessary information for training.

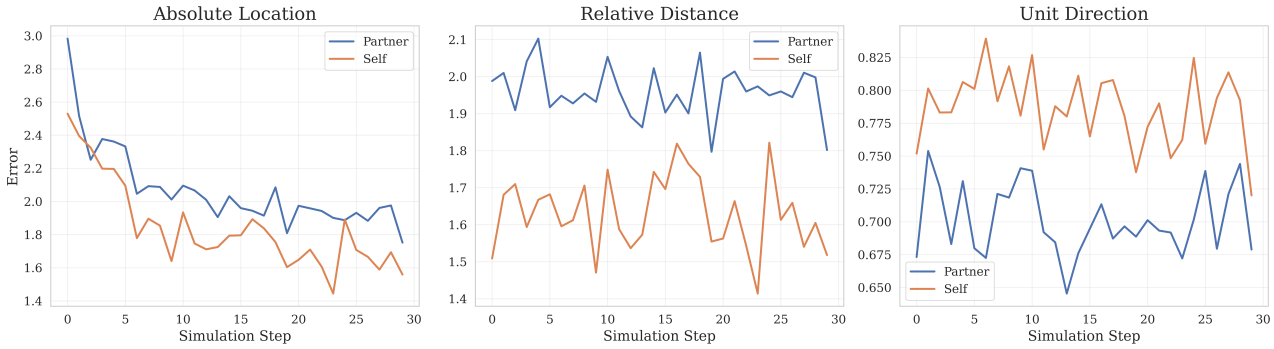


Figure 4.7: LSTM Probing Models Test Loss over simulation runs with various tasks.

The relative distance and unit direction models are shown to never improve over the course of a simulation, suggesting the LSTM does not follow egocentric memory. However, the absolute location model shows that the LSTM uses allocentric memory.

Even though the absolute location model shows the LSTM storing some information, it converges to an MSE of 1.6. This error represents an uncertainty radius of 1.6 cells, showing how the memory acts as a 'regional beacon' for the agent's targets.

An important conclusion from Figure 4.7 is that the agent's need for communication decreases as the simulation runs, and they store target locations in the LSTM module.

4.6 Semantic Grounding and Pragmatics

4.6.1 Semantic Grounding: $P(\mathbf{Target}^{t}_{\mathbf{partner}}|\mathbf{Code}^{t}_{self})$

Based on the conclusion from LSTM Probing (Section 4.5), less communication is required as the simulation progresses. Therefore, to analyse the semantic meaning of messages, only the first six simulation steps are used.

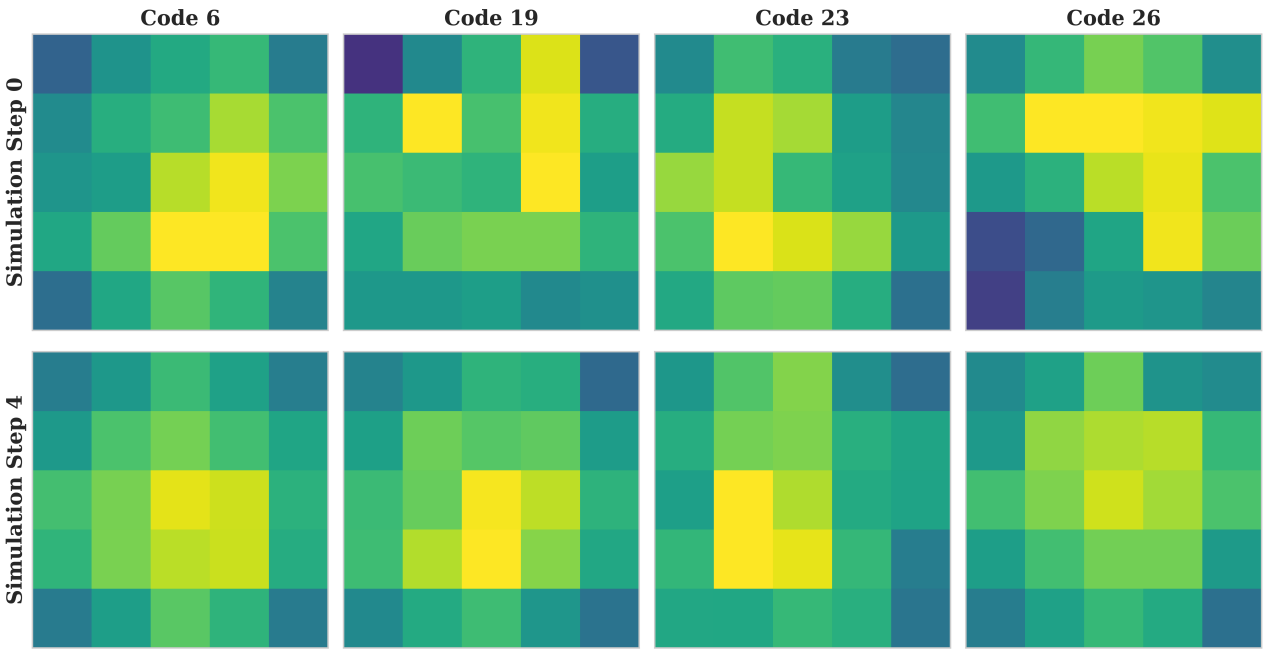


Figure 4.8: Normalised likelihood (P(target location | partner codes)) at simulation steps 0 and 4. Each column shows one code - the colour intensity (blue low, yellow high) reflects frequency across rollouts.

The likelihood of the location of the target given the code broadcast from the partner is demonstrated in Figure 4.8. It demonstrates that certain codes correspond to specific regions. The most prominent is code 6, which suggests the target is in the bottom right quadrant. This figure further consolidates the loss of communication during a simulation run. At the initial simulation step, target locations are clear; however, as the simulation progresses, even to step 4, the uncertainty in target locations given certain codes increases.

4.6.2 Causal Influence on Policy: $P(\mathbf{Action}^{t+1}_{partner}|\mathbf{Code}^{t}_{self})$

To see if the agent had a direct impact on the action of its partner, a confusion matrix is made:

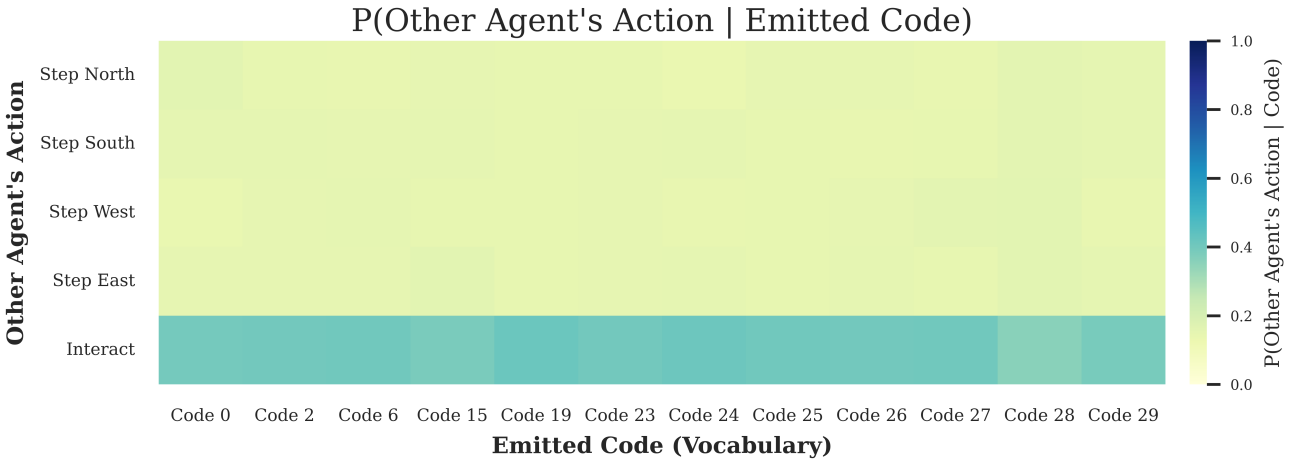


Figure 4.9: Probability of partner's action given received code within first four simulation steps.

As shown in Figure 4.9, calculating the conditional probability of the receiver's next action given the emitted code reveals a relatively uniform distribution across the action space. These findings further confirm those from the LSTM Probing (Section 4.5): the codes purely communicate allocentric information. As each agent independently navigates to its target based on its own knowledge of the environment, the actions taken follow a uniform distribution.

4.6.3 Context-Aware Signalling (Inventory-Dependent Language)

To determine whether the agents are using context-dependent signalling, the conditional probability of a token's use given its ingredients is calculated.

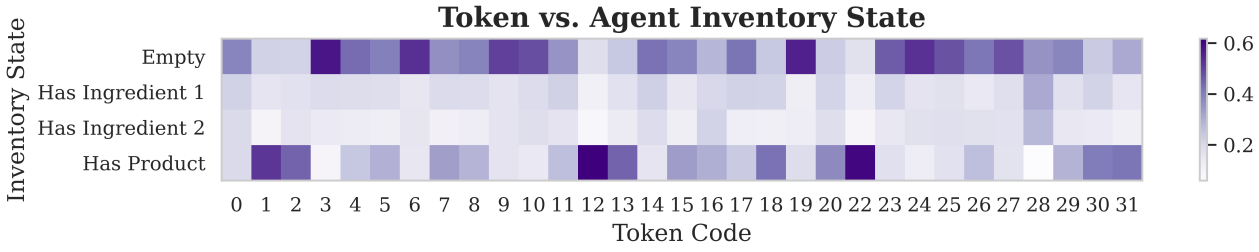
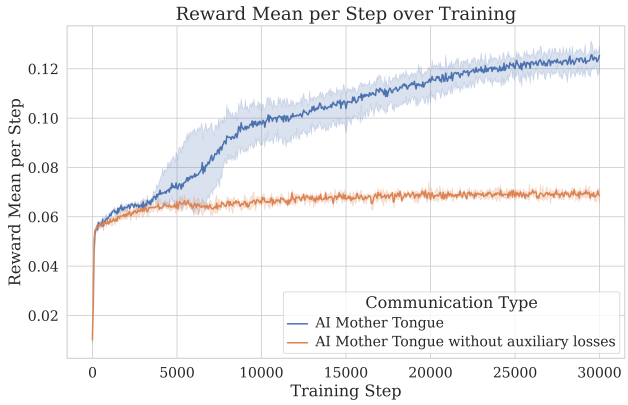


Figure 4.10: Probability of broadcasting token given inventory.

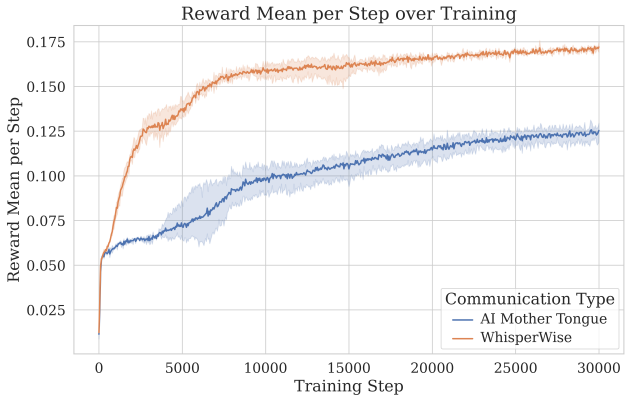
As shown in Figure 4.10, the active vocabulary undergoes large changes depending on the specific items currently held in the agent's inventory. When the agent has an empty inventory, the vocabulary is highly concentrated on a specific subset of codes. However, as the progression continues, the vocabulary immediately supersedes these widely used codes. Once the inventory contains the product, the vocabulary uses a concise set of codes likely used to coordinate the drop-off location. These results are consistent with context-dependent signalling.

4.7 AI Mother Tongue Ablation Studies

To determine the individual effects of the frozen encoder and the reflection strategies, two ablation studies were performed.



(a) Reflection vs no reflection.



(b) WhisperWise vs AI Mother Tongue.

Figure 4.11: Reward mean per step over 30,000 training steps. Each experiment was trained across three runs, with a 95% confidence interval shown. 4.11a compares reflection against no reflection; 4.11b compares the AI Mother Tongue architecture with the WhisperWise communication protocol.

Each component of the AI Mother Tongue is analysed to examine its effects on the mean task reward per step (Figure 4.11).

Before conducting a statistical comparison for Figure 4.11a, the assumptions underlying the t-test were evaluated. Shapiro-Wilk tests indicated both protocols were approximately normally distributed, and Levene's test indicated the variances were homogeneous. Given these conditions, a Welch's two-sample t-test was performed, revealing a significant difference between the protocols, $t = 19.45, p < 0.001$. This shows AI Mother Tongue (mean=0.1235) outperformed the AI Mother Tongue without auxiliary losses (mean=0.0704).

A statistical comparison was completed for Figure 4.11b. The same assumptions were evaluated using Shapiro-Wilk tests and Levene's test, which showed results similar to those before. Welch's t-test was performed and revealed a significant difference between the protocols, $t = -31.75, p < 0.001$. In this case, WhisperWise (mean=0.1729) performed better than the AI Mother Tongue architecture (mean=0.1236). These results suggest that the reflection strategies contribute more to successful emergent communication than the semantic bottleneck.

4.8 Scaling WhisperWise and AI Mother Tongue

The results of Section 4.7 suggest the AI Mother Tongue architecture relies primarily on the reflection strategies for its success. However, the original evaluation is restricted to a task that is relatively complex compared to Liu's, but ultimately simple. Therefore, it remains unclear whether either protocol can support emergent communication as the underlying POMDP becomes more complex.

4.8.1 Environment Setup

A more demanding task is set up for a more comprehensive evaluation of AI Mother Tongue and WhisperWise.

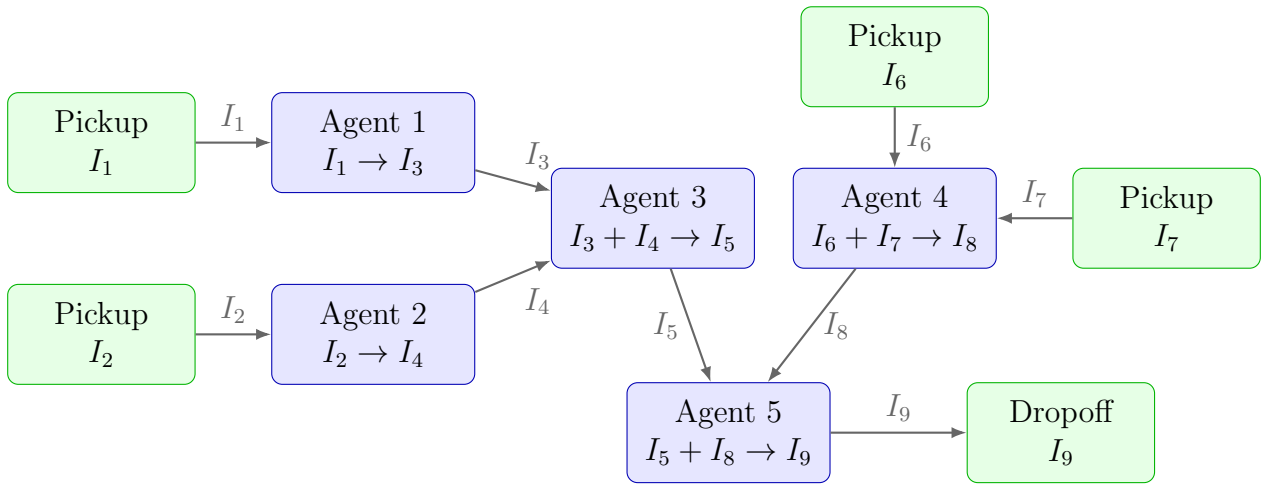


Figure 4.12: Agent-station Workflow.

The workflow is described in Figure 4.12. Each agent cannot observe their own pick-up stations. Each agent can view other agents within two cells and other stations within five cells.

The world is defined as a 15x15 grid. To reduce the difference in mean task reward per step between the communication and no-communication protocols, the number of actions agents are allowed to take in each world is limited to 200.

4.8.2 Task Performance

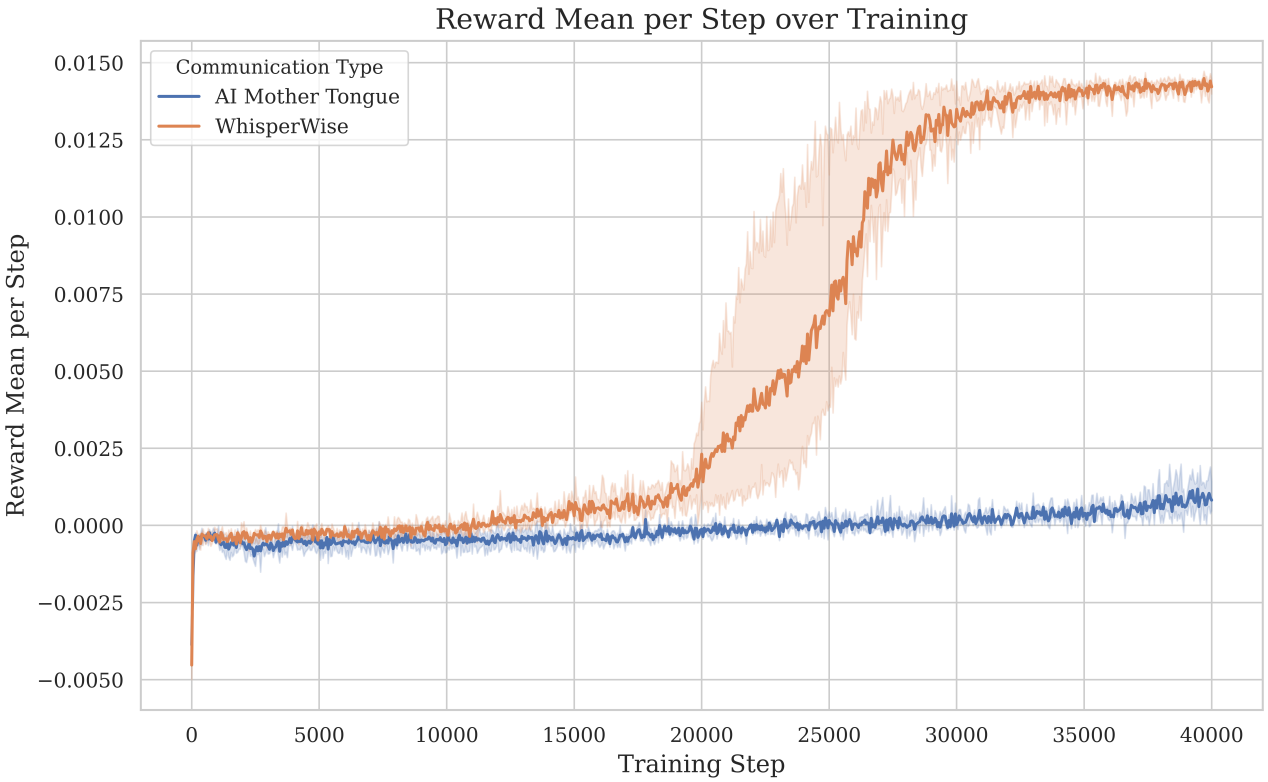


Figure 4.13: Reward mean per step over 40,000 PPO training steps. Each protocol was trained across three runs. A 95% confidence interval is shown.

As shown in Figure 4.13, WhisperWise outperformed AI Mother Tongue in this complex task. To further this, another Welch's two-sample t-test was performed. As in previous analyses, the assumptions for parametric testing were verified and satisfied. A significant difference between the two protocols was found, $t = -19.98, p = 0.0023$. WhisperWise achieved a higher mean score of 0.0141 than AI Mother Tongue's 0.0007.

4.8.3 WhisperWise Linguistic and Memory Analysis

To understand how the WhisperWise agents represent the environment, LSTM probing from Section 4.5 is re-applied.

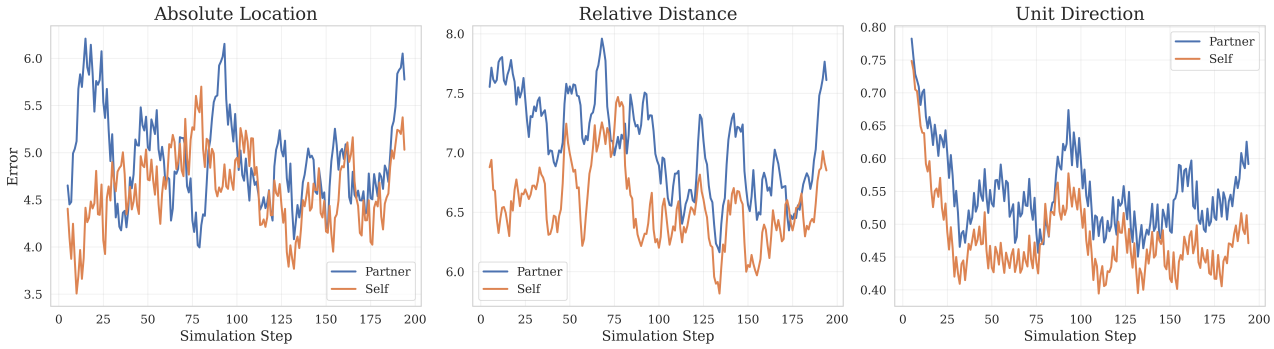


Figure 4.14: LSTM Probing Models Test Loss over simulation runs with various tasks.

In contrast to the AI Mother Tongue architecture, which improved only allocentric memory over time, the WhisperWise agents' policy learned a strong egocentric representation. The unit direction of the agents' own target and others' targets, decoded from the final LSTM layer, improved as the agents discovered stations.

In addition, WhisperWise agents continued to exhibit low-entropy communication, similar to that in the base task (see Appendix G for further results).

4.9 Limitations of Evaluation

Whilst the empirical evaluations presented in this chapter demonstrate statistically significant performance gains for the WhisperWise protocol, it is necessary to contextualise these findings within the physical compute constraints. In Deep Reinforcement Learning, multi-agent policies are highly sensitive to initial random seeds [61]. Even though the simulation was optimised, the computational overhead could exceed 17 hours of continuous GPU time (NVIDIA RTX 5060 Ti) per complete training run. As a result, the sample size for training runs was limited to three random seeds per protocol.

To compensate for this limitation, variance within each run was rigorously assessed by evaluating policies across 960 randomised parallel environments at every simulation step. This large-scale evaluation substantially reduces the impact of environmental stochasticity and provides a robust estimate of expected performance. This mirrors common RL practice of using many parallel environments for low-variance evaluations within fixed computational budgets [14].

Notably, the performance gap between WhisperWise and AI Mother Tongue was sufficiently large that their 95% confidence intervals did not overlap.

Therefore, whilst these results provide compelling evidence of WhisperWise's effectiveness, they should be interpreted as strong proof of concept.

4.10 Evaluation of Success Criteria

C1 - High-Throughput Environment C1 required a configurable, vectorised Rust environment with Python bindings, achieving at least 50K environment steps per second and supporting real-time visualisations. Section 3.2 detailed how this was implemented. The final system reached over 200K environment steps per second on my hardware, as shown in Section 4.2.1. C1 was fully achieved.

C2 - MARL Framework C2 required a functioning multi-agent reinforcement learning implementation with a shared actor and a centralised critic capable of learning the complex environment. Section 3.3 detailed how this was implemented using MAPPO. The final system with no communication reached $\sim$ 40% of the expert agent's mean task reward, as shown in section 4.3. C2 was fully achieved.

C3 - Baseline Communication Protocols C3 requires at least two protocols (no communication and discrete communication), which are interchangeable within the same training pipeline. The communication strategy pattern (as detailed in Section 3.3.5) demonstrated that protocols can be interchanged easily. The MARL baseline comparison in Section 4.3 could not be implemented without the hot-swapping of protocols. C3 was fully achieved.

C4 - AI Mother Tongue Architecture C4 required extending the AI Mother Tongue Architecture and performing a quantitative evaluation of the communication protocol, including achieving a Gini coefficient of at least 0.7, demonstrating a low-entropy active vocabulary, and showing a Zipfian distribution in codebook utilisation. C4 also states that the communication must exhibit context-aware signalling, verified by conditional probability distributions (e.g., $P(\text{Target}|\text{Code})$). The evaluation (Sections 4.4 - 4.6) details in-depth evaluations of communication perplexity, LSTM probing, and conditional distributions, in line with C4. A low entropy score of 2.98 and a high Gini coefficient of 0.78 were achieved. C4 was largely satisfied: high Gini coefficients, low-entropy vocabularies and heavy-tailed code usage, and conditional distributions consistent with context-aware signalling in this environment were observed.

C5 - Software Engineering and Reproducibility C5 required the framework to be reusable for other researchers. This includes high testing coverage, logging, and checkpointing. Sections 3.1.3 and 3.8 detail this implementation. The evaluation would not have been possible without this toolkit. C5 is well-supported: the framework has high unit-test coverage in both Rust and Python, structured configuration, logging and checkpointing, and open-source release with documentation intended to support reuse.

Chapter 5

Conclusions

Multi-agent reinforcement learning has potential in many real-world domains, such as robotic surgery, traffic optimisation, and search-and-rescue. To succeed in these partially observable environments, agents must learn to communicate effectively. However, replacing continuous communication with interpretable, discrete tokens introduces the "Joint Exploration Problem". This dissertation investigated whether the AI Mother Tongue architecture could overcome this hurdle within a complex MAPPO training pipeline. Whilst the architecture proved effective in simpler scenarios, it struggled to scale to more demanding tasks. In contrast, WhisperWise successfully scales to these complex environments, producing communication.

5.1 Work Completed

First, I implemented a configurable vectorised, high-throughput MARL environment capable of achieving over 100K steps per second. The simulation modelled a discrete two-dimensional environment populated by agents and stations. A lightweight real-time visualisation was created to enable inspection of the rollout phase.

Second, a MAPPO-based MARL training script under a centralised training, decentralised execution framework was implemented. It uses a shared actor, a global critic, an LSTM module, a custom rollout buffer for efficient storage and GAE calculations, and a communication protocol interface that supports hot-swapping. To reduce training time, a separate script was introduced to pre-train the actor-critic networks using imitation learning.

Third, I faithfully followed and extended the AI Mother Tongue architecture [6]. Pre-trained VQ-VAE and SQ-VAE (and an exploratory HQ-VAE variant, detailed in Appendix E) were trained on optimal rollout phases, injected into the actor network, and frozen before MARL training. To establish rigorous baselines and test novel improvements, alternative protocols were also developed, including Discrete Communication, No Communication and WhisperWise.

Fourth, a quantitative evaluation of communication protocols was performed. The analysis compared task reward across the communication protocols. This directly answered the research question proposed at the start of this dissertation.

Finally, the semantics of communication and memory were analysed. By collecting and visualising conditional distributions, such as the probability of a partner's target given a code and the probability of a partner's next action given a code, the dissertation examined whether the message correlates with goals or with the next action. This analysis showed the agent's communication carried interpretable meaning.

Ultimately, the evaluation suggests that the AI Mother Tongue architecture enables semantically meaningful discrete communication, outperforming both on-policy discrete and

no-communication baselines by overcoming the Communication Vacuum Equilibrium in simple tasks. However, the evidence indicates the AI Mother Tongue architecture overcomplicates the solution to the "Joint Exploration Problem". Consequently, this dissertation presents WhisperWise: a new, streamlined protocol that merges on-policy discrete communication with reflection strategies. For basic tasks, WhisperWise significantly outperforms the AI Mother Tongue architecture (reward mean per step improvement of $\sim$ 40%; see Section 4.8) and achieves a steeper, more efficient semantic compression. WhisperWise performed better in the tested scaling task, where AI Mother Tongue had limited success. These results suggest that reflection strategies may account for a greater share of the performance gain than the semantic bottleneck.

5.2 Lessons Learned

I have learned the importance of logging and checkpointing; when running a program that takes over seventeen hours to complete, waiting until the end to find it overfitted hours ago is catastrophic without these systems in place. In terms of MARL communication, I learned that the complexity of an emergent language is fundamentally bounded by the environment that necessitates it. Agents will reliably find the most compressed, 'lazy' communication strategy that satisfies the reward function. I also learned the ethical weight of the "black box" nature of MARL communication. Relying on non-human-readable protocols in safety-critical domains risks serious failures if the system encounters unseen states. The analysis tools produced take a step further towards AI transparency, reducing this trust deficit.

5.3 Future Work

Each agent's code is broadcast to all other agents. For simulations with more than two agents, this will inevitably lead to agents listening to code that is not relevant to their policy. An improvement would be applying an attention mechanism to determine which codes are sent to which agents, similar to TarMAC [22]. This attention mechanism has another key potential use. When an agent observes the environment, they only see the closest visible entity. This could be improved by using an attention mechanism to select which visible entity to observe.

Another direction for improvement is to evaluate whether the established communication protocols can scale to highly complex, hierarchical environments (such as 50x50 grids with agents passing ingredients to one another). This would, in theory, push the HQ-VAE to establish macro- and micro-strategies (Appendix E).

Finally, a third direction is to further research the WhisperWise protocol. WhisperWise (as addressed in Section 4.9) requires evaluation over a significantly larger range of random seeds ($\geq$ 30). WhisperWise's next step is to be integrated into a real-world application. This transition necessitates a rigorous data protection framework. For example, the expert agent would likely be human, triggering the application of strict data protection laws (such as GDPR [62]).

In conclusion, this work shows that the AI Mother Tongue architecture overcomplicates the solution to the "Joint Exploration Problem". The reflection strategies teach agents to use the communication channel effectively, but the semantic bottleneck impedes their learning. This dissertation introduces WhisperWise, a scalable and elegant solution that takes a step closer to developing artificial agents that can coordinate effectively and transparently.

Bibliography

[1] David Silver et al. "Mastering the game of Go with deep neural networks and tree search". In: *Nature* 529.7587 (Jan. 2016), pp. 484–489. DOI: 10.1038/nature16961. URL: https://www.nature.com/articles/nature16961.

[2] Kaiqing Zhang, Zhuoran Yang, and Tamer Başar. *Multi-Agent Reinforcement Learning: A Selective Overview of Theories and Algorithms*. 2019. eprint: arXiv:1911.10635.

[3] Changxi Zhu, Mehdi Dastani, and Shihan Wang. "A Survey of Multi-Agent Deep Reinforcement Learning with Communication". In: (2022). eprint: arXiv:2203.08975.

[4] Francisco S. Melo, Matthijs T. J. Spaan, and Stefan Witwicki. "POMDP-based Communication in Multiagent Systems". In: *Proceedings of the 11th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2012)*. IFAAMAS, 2012, pp. 1029–1036. URL: https://www.st.ewi.tudelft.nl/~mtjspaan/pub/Melo12eumasPost.pdf.

[5] Tom Eccles et al. *Biases for Emergent Communication in Multi-agent Reinforcement Learning*. 2019. eprint: arXiv:1912.05676.

[6] Hung Ming Liu. "AI Mother Tongue: Self-Emergent Communication in MARL via Endogenous Symbol Systems". In: *arXiv preprint arXiv:2507.10566* (2025). URL: https://arxiv.org/abs/2507.10566.

[7] EmbersArc. *Best Practices when training with PPO*. https://github.com/EmbersArc/PPO/blob/master/best-practices-ppo.md. PPO hyperparameter tuning guide. 2017.

[8] Frans A. Oliehoek and Christopher Amato. *A Concise Introduction to Decentralized POMDPs*. Vol. 9. SpringerBriefs in Intelligent Systems. Springer, 2016. ISBN: 978-3-319-53778-4. DOI: 10.1007/978-3-319-53779-1. URL: https://dblp.org/pers/bd/o/Oliehoek:Frans_A=.

[9] Jakob N. Foerster et al. "Counterfactual Multi-Agent Policy Gradients". In: *Proceedings of the Thirty-Second AAAI Conference on Artificial Intelligence*. AAAI Press, 2018, pp. 2976–2984. URL: https://arxiv.org/abs/1705.08926.

[10] Georgios Papoudakis et al. *Benchmarking Multi-Agent Deep Reinforcement Learning Algorithms in Cooperative Tasks*. 2020. eprint: arXiv:2006.07869.

[11] Peter Sunehag et al. *Value-Decomposition Networks For Cooperative Multi-Agent Learning*. 2017. eprint: arXiv:1706.05296.

[12] Tabish Rashid et al. *QMIX: Monotonic Value Function Factorisation for Deep Multi-Agent Reinforcement Learning*. 2018. eprint: arXiv:1803.11485.

[13] Ryan Lowe et al. *Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments*. 2017. eprint: arXiv:1706.02275.

[14] Chao Yu et al. *The Surprising Effectiveness of PPO in Cooperative, Multi-Agent Games*. 2021. eprint: arXiv:2103.01955.

[15] John Schulman et al. *Proximal Policy Optimization Algorithms*. 2017. eprint: arXiv:1707.06347.

[16] John Schulman et al. *Trust Region Policy Optimization*. 2015. eprint: arXiv:1502.05477.

[17] Christopher Amato. *An Introduction to Centralized Training for Decentralized Execution in Cooperative Multi-Agent Reinforcement Learning*. 2024. eprint: arXiv:2409.03052.

[18] EmergentMind. *Centralized Training with Decentralized Execution (CTDE) with MAPPO*. https://www.emergentmind.com/topics/centralized-training-decentralized-execution-ctde-with-mappo. Multi-agent RL architecture overview. Feb. 2026.

[19] John Schulman et al. *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. arXiv:1506.02438. Version 6, last revised 20 Oct 2018. 2015.

[20] Shivang Shrivastav. *Generalized Advantage Estimation (GAE): A Deep Dive into Bias, Variance, and Policy Gradients*. Medium. Accessed: 2026-04-02. 2025. URL: https://shivang-ahd.medium.com/generalized-advantage-estimation-a-deep-dive-into-bias-variance-and-policy-gradients-a5e0b3454dad.

[21] Jakob N. Foerster et al. *Learning to Communicate with Deep Multi-Agent Reinforcement Learning*. 2016. eprint: arXiv:1605.06676.

[22] Abhishek Das et al. *TarMAC: Targeted Multi-Agent Communication*. 2018. eprint: arXiv:1810.11187.

[23] Mycal Tucker et al. *Emergent Discrete Communication in Semantic Spaces*. 2021. eprint: arXiv:2108.01828.

[24] Paul J. Pritz and Kin K. Leung. *Belief States for Cooperative Multi-Agent Reinforcement Learning under Partial Observability*. 2025. eprint: arXiv:2504.08417.

[25] Claude E. Shannon. "A Mathematical Theory of Communication". In: *Bell System Technical Journal* 27.3 (1948), pp. 379–423, 623–656. DOI: 10.1002/j.1538-7305.1948.tb01338.x.

[26] Aaron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. *Neural Discrete Representation Learning*. 2017. eprint: arXiv:1711.00937.

[27] Yuhta Takida et al. *SQ-VAE: Variational Bayes on Discrete Representation with Self-annealed Stochastic Quantization*. 2022. eprint: arXiv:2205.07547.

[28] Yuhta Takida et al. *HQ-VAE: Hierarchical Discrete Representation Learning with Variational Bayes*. 2023. eprint: arXiv:2401.00365.

[29] Institute of Project Management. *Agile Methodology: The Complete Guide for 2026*. Institute of Project Management. Mar. 2026. URL: https://instituteprojectmanagement.com/blog/agile-methodology/ (visited on 04/20/2026).

[30] NumPy Developers. *NumPy*. https://numpy.org. Version 2.4. Dec. 2025.

[31] pandas developers. *pandas: powerful Python data analysis and manipulation tools*. https://pandas.pydata.org. Version 3.0.1. Feb. 2026.

[32] Matplotlib Development Team. *Matplotlib: Visualization library for Python*. https://matplotlib.org. Version 3.10. 2026.

[33] Project Jupyter. *Jupyter Notebook*. https://jupyter.org. Version 7.5. 2026.

[34] PyTorch Foundation. *PyTorch*. https://pytorch.org. Version 2.7. 2026.

[35] Python Software Foundation. *GlobalInterpreterLock*. https://wiki.python.org/moin/GlobalInterpreterLock. (Visited on 04/18/2026).

[36] The rayon-rs Developers. *Rayon: A data parallelism library for Rust*. https://github.com/rayon-rs/rayon. 2016.

[37] Microsoft. *TypeScript*. https://www.typescriptlang.org. JavaScript with syntax for types. 2026.

[38] p5.js Contributors. *p5.js*. https://p5js.org. A free and open-source JavaScript library for creative coding. 2026.

[39] Tokio Contributors. *Tokio: An asynchronous runtime for Rust*. https://tokio.rs. Version 1.40. 2026.

[40] Sean McArthur and warp contributors. *warp: A super-easy, composable, web server framework for warp speeds*. https://github.com/seanmonstar/warp. Version 0.3. 2026.

[41] GitHub, Inc. *GitHub*. https://github.com. Accessed: 2026-04-18. 2026.

[42] PyO3 Project. *PyO3 - Rust bindings for the Python interpreter*. https://github.com/PyO3/pyo3. 2017. (Visited on 04/10/2026).

[43] PyO3 Project. *setuptools-rust*. https://pypi.org/project/setuptools-rust/. 2021. (Visited on 04/10/2026).

[44] Refactoring Guru. *Adapter*. Refactoring.Guru. 2024. URL: https://refactoring.guru/design-patterns/adapter (visited on 04/10/2026).

[45] Gene M Amdahl. "Validity of the single processor approach to achieving large scale computing capabilities". In: *Proceedings of the April 18–20, 1967, spring joint computer conference* (1967), pp. 483–485.

[46] Evan You and Vite Contributors. *Vite: Next Generation Frontend Tooling*. https://vite.dev. Version 8.0. 2026.

[47] Python Software Foundation. *Python Lists: Dynamic Arrays*. https://docs.python.org/3/c-api/list.html. Over-allocation and resizing via doubling capacity. 2026.

[48] Stefano Borini. *Python and memory fragmentation*. http://stefanoborini.com/python-and-memory-fragmentation/. 2014.

[49] John Schulman et al. *High-Dimensional Continuous Control Using Generalized Advantage Estimation*. https://arxiv.org/abs/1506.02438. GAE with advantage normalisation for stable policy gradients. 2015.

[50] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. *Estimating or Propagating Gradients Through Stochastic Neurons for Conditional Computation*. arXiv:1308.3432. Version 1. Aug. 2013.

[51] Minyoung Huh et al. *Straightening Out the Straight-Through Estimator: Overcoming Optimization Challenges in Vector Quantized Networks*. 2023. eprint: arXiv:2305.08842.

[52] Tim R. Davidson et al. "Hyperspherical Variational Auto-Encoders". In: (2018). eprint: arXiv:1804.00891.

[53] Eric Jang, Shixiang Gu, and Ben Poole. *Categorical Reparameterization with Gumbel-Softmax*. 2016. eprint: arXiv:1611.01144.

[54] Chris J. Maddison, Andriy Mnih, and Yee Whye Teh. "The Concrete Distribution: A Continuous Relaxation of the Categorical Distribution". In: *Advances in Neural Information Processing Systems 30 (NeurIPS 2017)*. Ed. by I. Guyon et al. Curran Associates, Inc., 2017, pp. 2528–2537. URL: https://arxiv.org/abs/1611.00712.

[55] Neil C. Rabinowitz et al. "Machine Theory of Mind". In: *Proceedings of the 35th International Conference on Machine Learning (ICML 2018)*. PMLR, 2018, pp. 4218–4227. URL: http://proceedings.mlr.press/v80/rabinowitz18a.html.

[56] David Premack and Guy Woodruff. "Does the chimpanzee have a theory of mind?" In: *Behavioral and Brain Sciences* 1.4 (1978), pp. 515–526. DOI: 10.1017/S0140525X00076512.

[57] Will McGugan and Textualize. *Rich: A Python library for rich text and beautiful formatting in the terminal*. https://github.com/Textualize/rich. Version 13.9. 2026.

[58] George Kingsley Zipf. *Human Behavior and the Principle of Least Effort: An Introduction to Human Ecology*. Cambridge, Massachusetts: Addison-Wesley Press, 1949.

[59] Kwame Boamah-Addo, Tomasz J. Kozubowski, and Anna K. Panorska. "A discrete truncated Zipf distribution". In: *Statistica Neerlandica* 77.2 (2023), pp. 156–187. DOI: 10.1111/stan.12280. URL: https://onlinelibrary.wiley.com/doi/epdf/10.1111/stan.12280.

[60] Guillaume Alain and Yoshua Bengio. *Understanding intermediate layers using linear classifier probes*. 2016. eprint: arXiv:1610.01644.

[61] Peter Henderson et al. *Deep Reinforcement Learning that Matters*. 2017. eprint: arXiv:1709.06560.

[62] European Union. *General Data Protection Regulation (GDPR)*. Regulation (EU) 2016/679. 2016. URL: https://gdpr-info.eu (visited on 05/02/2026).

[63] Bin Dai, Ziyu Wang, and David Wipf. *The Usual Suspects? Reassessing Blame for VAE Posterior Collapse*. 2019. eprint: arXiv:1912.10702.

Appendix A

Multi-Agent Proximal Policy Optimisation (MAPPO) Pseudocode

Algorithm 1 MAPPO Minibatch Update.

Require: Buffer minibatches, Clip hyperparameter ϵ, Value coefficient c_{vf}, Entropy coefficient c_{ent}

for batch $\in$ buffer **do**
  Extract: Observations O, O_{global}, Actions a, Comms c, Advantage A, Returns $\hat{R}$
  Extract: Old log probs $\log \pi_{\theta_{\text{old}}}(a), \log \pi_{\phi_{\text{old}}}(c)$, Initial LSTM states h_0, c_0
  ▷ **Critic Evaluation**
  $V \leftarrow V_\psi(O_{global})$ ▷ Compute new state values
  $L_{\text{critic}} \leftarrow \text{MSE}(V, \hat{R})$
  ▷ **Communication Embedding & Shifting**
  $C_{\text{cont}} \leftarrow \text{EmbedCodebooks}(c)$ ▷ Convert discrete comms to continuous embeddings
  $C_{\text{shifted}} \leftarrow \text{ShiftRight}(C_{\text{cont}}, \text{steps} = 1)$
  ▷ **Actor & Comm Protocol Forward Pass**
  $\pi_\theta(a), h' \leftarrow \text{ActorRNN}(O, h_0, c_0, C_{\text{shifted}})$
  $\pi_\phi(c), \mathcal{H}_c, L_{\text{proto}} \leftarrow \text{CommProtocol}(h', c, C_{\text{cont}}, \hat{R}, V)$
  ▷ **Action PPO Surrogate Loss**
  $r_a \leftarrow \exp(\log \pi_\theta(a) - \log \pi_{\theta_{\text{old}}}(a))$
  $L_{\text{action}} \leftarrow -\mathbb{E}\left[\min\left(r_a A, \text{clip}(r_a, 1-\epsilon, 1+\epsilon)A\right)\right]$
  ▷ **Communication PPO Surrogate Loss**
  $r_c \leftarrow \exp(\log \pi_\phi(c) - \log \pi_{\phi_{\text{old}}}(c))$
  $L_{\text{comm}} \leftarrow -\mathbb{E}\left[\min\left(r_c A, \text{clip}(r_c, 1-\epsilon, 1+\epsilon)A\right)\right]$
  ▷ **Distributions & Entropy**
  $\log \pi_\theta(a) \leftarrow \text{LogProb}(\pi_\theta, a)$
  $\mathcal{H}_a \leftarrow \text{Entropy}(\pi_\theta)$
  $L_{\text{ent}} \leftarrow -\mathbb{E}[\mathcal{H}_a + \mathcal{H}_c]$
  ▷ **Total Loss Computation**
  $L_{\text{actor}} \leftarrow L_{\text{action}} + L_{\text{comm}} + L_{\text{proto}}$
  $L_{\text{total}} \leftarrow L_{\text{actor}} + c_{vf} L_{\text{critic}} + c_{ent} L_{\text{ent}}$
  ▷ **Optimisation Step**
  $\theta, \phi, \psi \leftarrow \text{ZeroGradients}()$
  Compute $\nabla_{\theta,\phi,\psi} L_{\text{total}}$
  $\text{ClipGradNorm}(\theta, 0.5)$; $\text{ClipGradNorm}(\psi, 0.5)$
  $\text{OptimiserStep}()$
end for

Appendix B

Generalised Advantage Estimation (GAE) Pseudocode

Algorithm 2 Compute Advantages and Returns using GAE.

procedure GeneralisedAdvantageEstimation($V_{last}, \gamma, \lambda$)
    Input: Last value V_{last}, discount factor γ, GAE parameter λ
    Initialise: $gae \leftarrow 0$
    $T \leftarrow$ timesteps_per_run
    for $t = T - 1$ **down to** 0 **do**
        $r \leftarrow R[t]$ ▷ Reward at timestep t
        $v \leftarrow V[t]$ ▷ Value at timestep t from critic
        $\delta \leftarrow r + \gamma V_{last} - v$ ▷ Compute 1-step TD Error
        $gae \leftarrow \delta + \gamma\lambda \cdot gae$ ▷ Compute GAE advantage
        $A[t] \leftarrow gae$ ▷ Store advantages
        $Returns[t] \leftarrow gae + v$ ▷ Store returns
        $V_{last} \leftarrow v$ ▷ Update last value for the next iteration $(t - 1)$
    end for
end procedure

Appendix C

Data-driven Codebook Initialisation Pseudocode

Algorithm 3 Data-driven Codebook Initialisation via K-Means.

procedure FitRQStream(X, Q, d)
  Input: Initial continuous latents/residuals X, Quantiser Q, latent dim d
  $X \leftarrow \text{Reshape}(X, [-1, d])$ ▷ Flatten batch and time dimensions
  $L \leftarrow Q.\text{rq\_levels}$
  $K \leftarrow \text{VocabSize}$
  for $l = 1$ **to** L **do**
    $X_{\text{norm}} \leftarrow \frac{X}{||X||_2 + \epsilon}$ ▷ L2 Normalise residuals ($\epsilon = 10^{-5}$)
    $C \leftarrow \text{MiniBatchKMeans}(X_{\text{norm}}, \text{clusters} = K)$ ▷ Find K centroids
    if Encoder is SQ-VAE **then**
      $C \leftarrow \frac{C}{||C||_2}$ ▷ Normalise centroids onto unit sphere
    end if
    $Q.\text{codebook}[l] \leftarrow C$ ▷ Set embedding weights
    $A \leftarrow \text{PredictClusters}(X_{\text{norm}}, C)$ ▷ Get cluster assignments
    $X \leftarrow X - C[A]$ ▷ Compute residuals for the next RQ level
  end for
end procedure

procedure SetPriors($M, \mathcal{D}$)
  Input: Model M, Dataloader $\mathcal{D}$
  $\mathcal{Z}_{\text{top}} \leftarrow [], \quad \mathcal{Z}_{\text{bot}} \leftarrow [], \quad \mathcal{Z} \leftarrow []$
  ▷ **Step 1: Collect Continuous Latents**
  for batch $O \in \mathcal{D}$ **do**
    $z \leftarrow M.\text{encoder.GetContinuous}(O)$
    Append z to $\mathcal{Z}$
  end for
  ▷ **Step 2: Fit Residual Quantisation Streams**
  $X \leftarrow \text{Concatenate}(\mathcal{Z})$
  FitRQStream($X, M.Q, d$)
end procedure

Appendix D

Variational Autoencoder (VAE) Ablation Studies

A poorly initialised frozen encoder could severely limit the agents' language, preventing them from reaching their full cooperation potential.

D.0.1 Discretisation Stability: VQ-VAE vs SQ-VAE

For this evaluation, reconstruction loss and commitment loss are tracked over a 50-epoch training process using 26880 datapoints. Liu defined a healthy encoder for the AI Mother Tongue system [6]. Reconstruction loss should fall between 0.08 and 0.15. Commitment Loss must remain below 1.0.

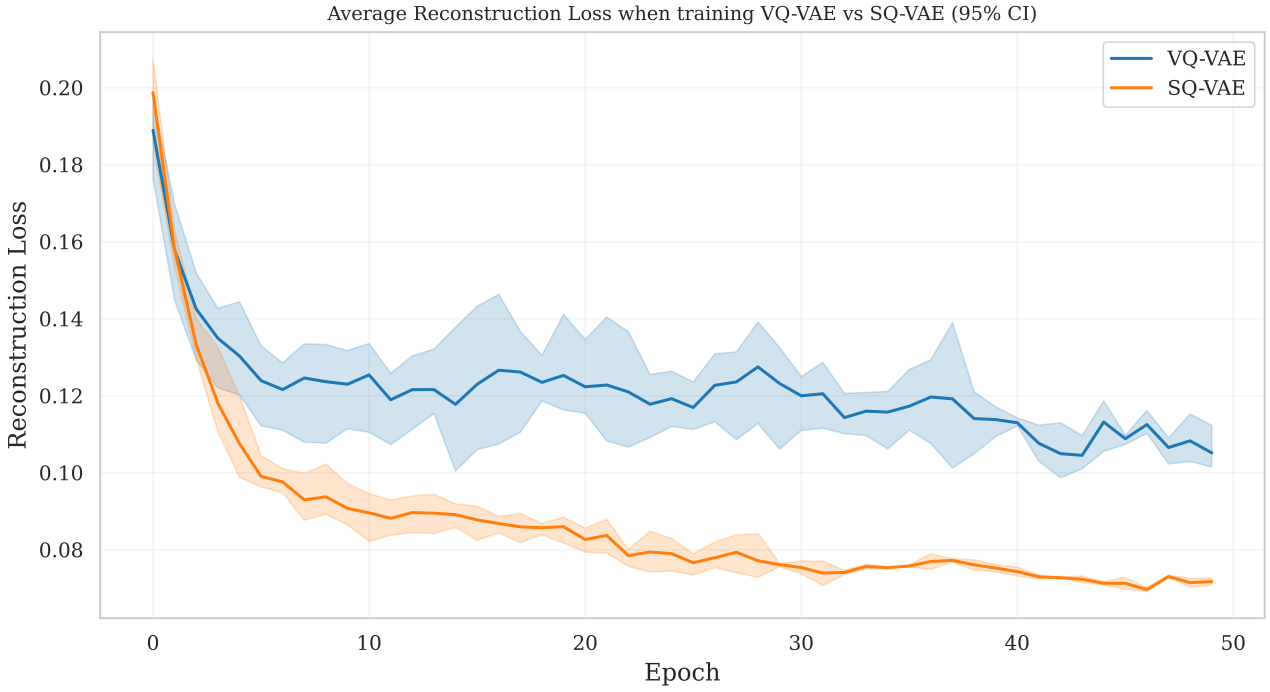


Figure D.1: Reconstruction Loss over 50 epochs comparing VQ-VAE and SQ-VAE.

As shown in Figure D.1, both methods of training got below the maximum reconstruction loss of 0.15. However, training with the SQ-VAE resulted in a final reconstruction below 0.08. Liu's recommended loss for a healthy codebook is based on low reconstruction error, which is often a symptom of 'posterior collapse' [26, 63]. This is where the network memorises a small set of generalisable states.

To determine if this 'posterior collapse' has occurred, the codebook utilisation was analysed.

	VQ-VAE	SQ-VAE
Codebook Utilisation	$66.67\% \pm 4.8\%$	$94.8\% \pm 6.5\%$

Table D.1: Codebook utilisation comparison between training with VQ-VAE and SQ-VAE.

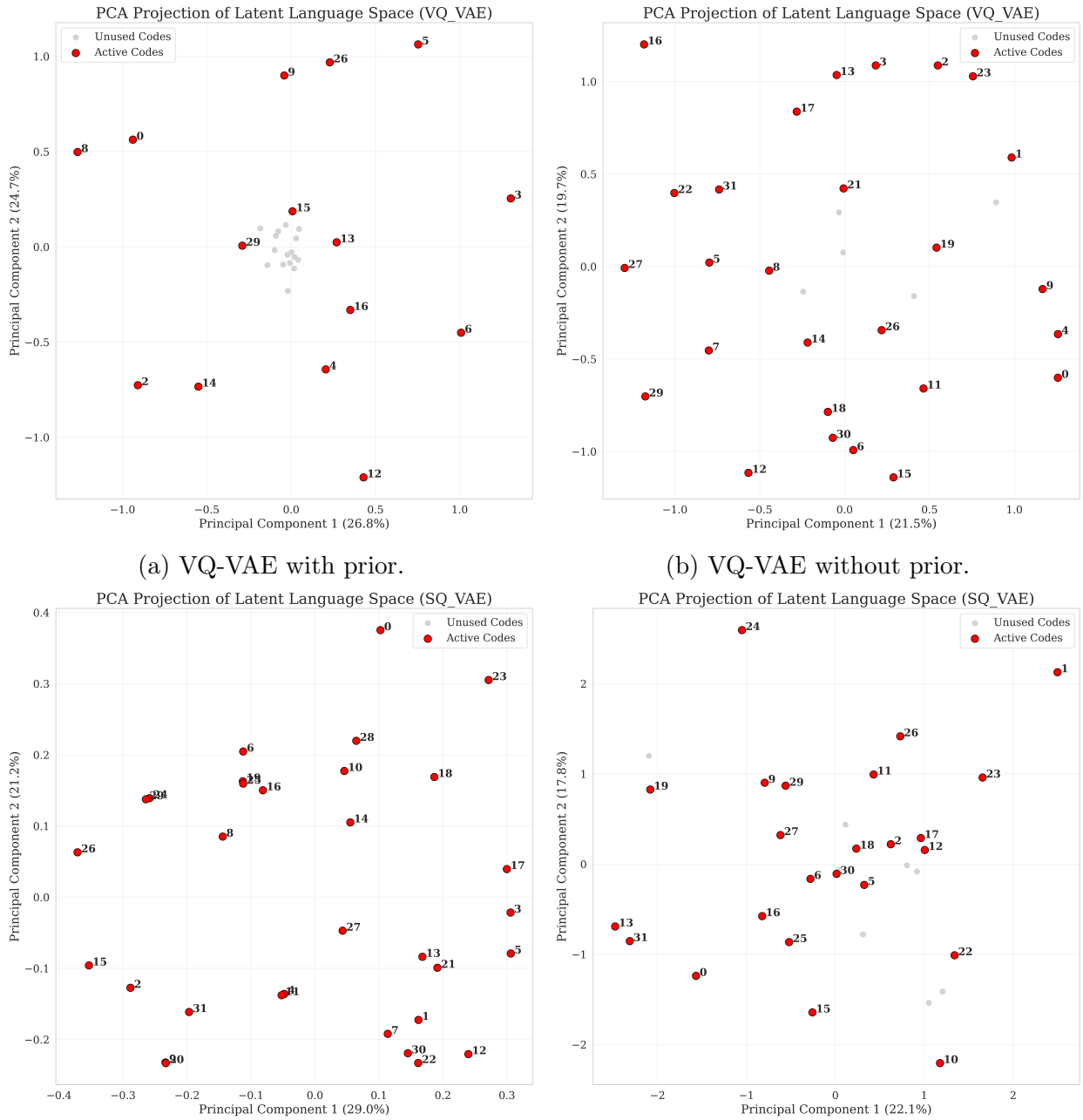
The table D.1 clearly shows that using an SQ-VAE, though having a low reconstruction loss, in fact has a better codebook utilisation than using a VQ-VAE. Therefore, this low loss indicates high model effectiveness rather than 'posterior collapse'.

By the end of training, the VQ-VAE's commitment loss had converged to ~ 0.25. This is well below Liu's requirement.

D.0.2 Codebook Initialisation

To evaluate the robustness of the VAEs, an ablation study is performed. Two initialisation strategies are compared: a standard uniform random initialisation, and a data-driven K-Means prior.

For the VQ-VAE, the uniform random prior yielded superior reconstruction loss and codebook utilisation compared to the K-Means prior. However, the SQ-VAE exhibited the opposite behaviour: the reconstruction loss and codebook utilisation were superior when using the K-Means prior rather than the random uniform prior.



(a) VQ-VAE with prior.

(b) VQ-VAE without prior.

(c) SQ-VAE with prior.

(d) SQ-VAE without prior.

Figure D.2: PCA projections of latent codes for SQ-VAE and VQ-VAE with and without the prior.

To visualise, Principal Component Analysis (PCA) was applied to reduce the dimensionality of the learned codebook embeddings, see Figure D.2. This visualisation explains the findings. For VQ-VAE (Figures D.2a and D.2b), the random uniform initialisation scattered the codes across the latent space. This allowed the untrained encoder to naturally interact with them, enabling gradients to flow through each embedding. Conversely, the K-Means prior forced the embeddings into a small region, where the untrained encoder would interact only with a select few edge embeddings, leaving a high concentration of unused codes.

For (vMF) SQ-VAE (Figures D.2c and D.2d), when using L_2-normalised Spherical K-Means prior, the embeddings were each set in the correct region and the model used its stochastic annealing purely for local fine-tuning. Whereas, when using the random initialisation,

embeddings frequently spawned at extreme angular distances from the true data distribution, and the model's annealing process pushed the embeddings into a suboptimal local minima.

Appendix E

Hierarchically-Quantised Variational Autoencoder (HQ-VAE)

E.1 Implementing HQ-VAE

Liu [6] suggested implementing Hierarchically-Quantised Variational Autoencoder (HQ-VAE) [28]. The idea behind this is to split the communication into a high-level strategy and a low-level strategy, where the low-level strategy is conditioned on the high-level strategy. By splitting the communication into macro-level and micro-level strategies, the model significantly reduces the computational burden associated with large, flat codebooks.

Common features are first extracted through a shared encoder:

$$h_{\text{shared}} = Enc_{\theta_E}(x) \tag{E.1}$$

The top level, responsible for macro-level strategies, projects these shared features to a vector:

$$z_{\text{top}} = MLP_{\text{top}}(h_{\text{shared}}) \tag{E.2}$$

The bottom level, responsible for micro-level strategies, projects these shared features conditioned on the top level to a vector:

$$z_{\text{bottom}} = MLP_{\text{bottom}}([h_{\text{shared}}, z_{\text{top}}]) \tag{E.3}$$

These vectors are quantised using one of our aforementioned methods (VQ-VAE, SQ-VAE).

$$z_{q,top} = \text{Quantiser}(z_{top}) \tag{E.4}$$

$$z_{q,bottom} = \text{Quantiser}(z_{bottom}) \tag{E.5}$$

Finally, the quantised vectors are concatenated to produce the final full communication vector z_q:

$$z_q = [z_{q,top}, z_{q,bottom}] \tag{E.6}$$

E.2 HQ-VAE Hyperparameters

The hyperparameters used for SQ-VAE and VQ-VAE are inappropriate for the AI Mother Tongue architecture with a discrete bottleneck using HQ-VAE. For this, each codebook is given a vocabulary size of 16 (giving an effective vocabulary size of 256), allowing different levels of detail in different codebooks. The code size also had to be increased from eight to sixteen. In HQ-VAE, the code size is split across the codebooks (in a 1:2 ratio), so a total code size of eight would allocate only two dimensions to the top-level latent. With vMF SQ-VAE at the top level, a 2-dimensional unit sphere cannot support 16 codes, leading to a collapsed top-level codebook. Using the code size of sixteen gives a sufficient five dimensions for the top-level codebook.

E.3 HQ-VAE Analysis

Replacing the single-level VAE bottleneck with an HQ-VAE did not significantly change task-level performance. HQ-VAE's goal was for the top-level codebook to capture macro-level strategies and the bottom-level codebook to capture micro-level strategies. To test this, I applied the same analyses as in Section 4.3 - 4.6 on each codebook and on the bottom-level codebook symbols conditioned on the top-level symbols. In all cases, no clear, distinguishable strategies emerged at either level. The lack of strategies in either codebook is likely due to the flat nature of the task (e.g. Find Station A). A hierarchical language may require a more hierarchical task.

Appendix F

Expected Number of Actions without Communication

To determine the location of a target which is not visible, the agent must be on top of the invisible entity and interact with it. In a 5x5 world, to search the entire grid takes 50 actions (25 movement and 25 interaction). For a single entity, it would be expected to find the entity after searching half the location ($(25 + 1)/2 = 13$ location, $= 26$ actions) Therefore, for two entities, the expectation of the larger number of locations searched for the two entities is calculated.

$$\mathbb{E}[\text{Maximum Location}] = \frac{2(25+1)}{2+1} = \frac{52}{3} \approx 17.33 \text{ locations} \tag{F.1}$$

So the number of actions on expectation is $(2 \times 17.33 =)$ 34.67.

Appendix G

WhisperWise Language Analysis on the Complex Task

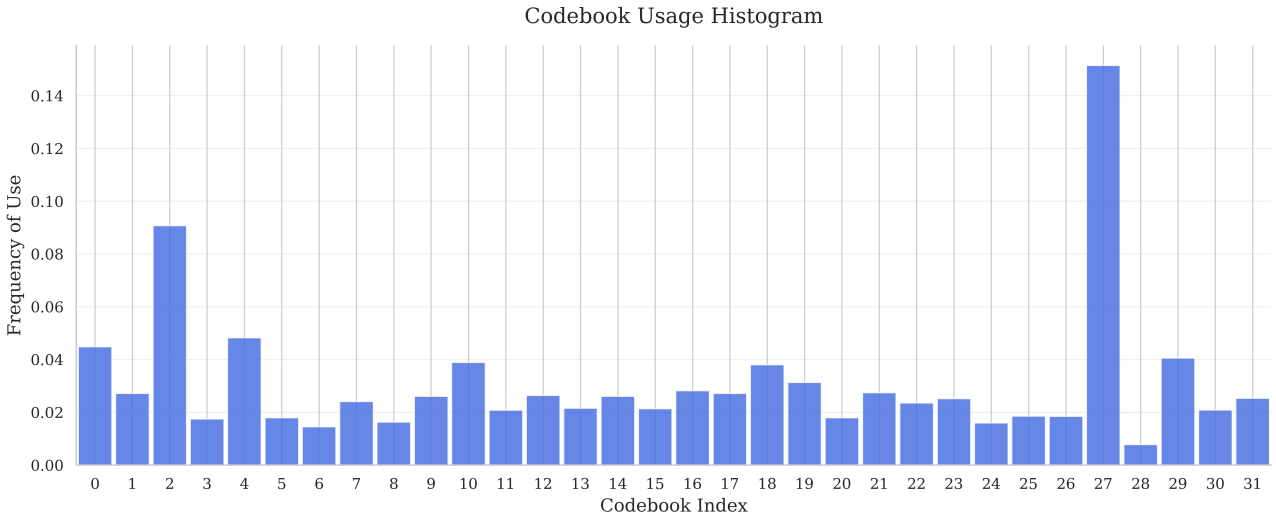


Figure G.1: Normalised frequency of each code across 2880 runs.

The agent's utilisation of the vocabulary gives an entropy score of 4.61 and a Gini coefficient of 0.94 (Figure G.1). This suggests the agent utilises the entire capacity of the codebook, unlike the AI Mother Tongue protocol.

The vocabulary usage closely follows Zipf's Law (Figure G.2) until code around code 31, where, as before, the truncated Zipfian distribution is exhibited.

The WhisperWise can learn to use the communication channel (Figure G.3), unlike the AI Mother Tongue protocol. The protocol applies semantic compression during the first 5000 timesteps and further refines the language to a minimal set by about 20000 timesteps. From then on, WhisperWise's communication protocol steadily increases, suggesting that the agents are learning to communicate the fine details of the environment. Future work should evaluate how this communication protocol's perplexity varies over time until convergence.

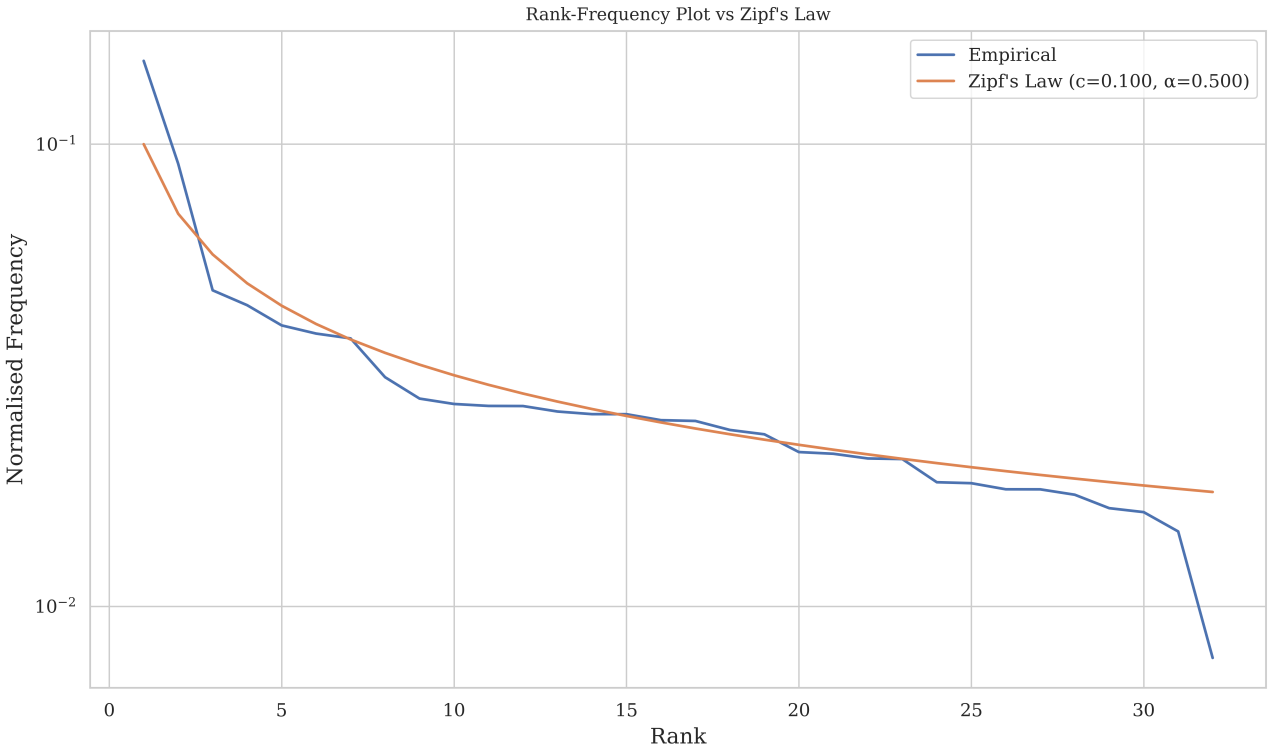


Figure G.2: Vocabulary rank-frequency plot compared to Zipf's Law.

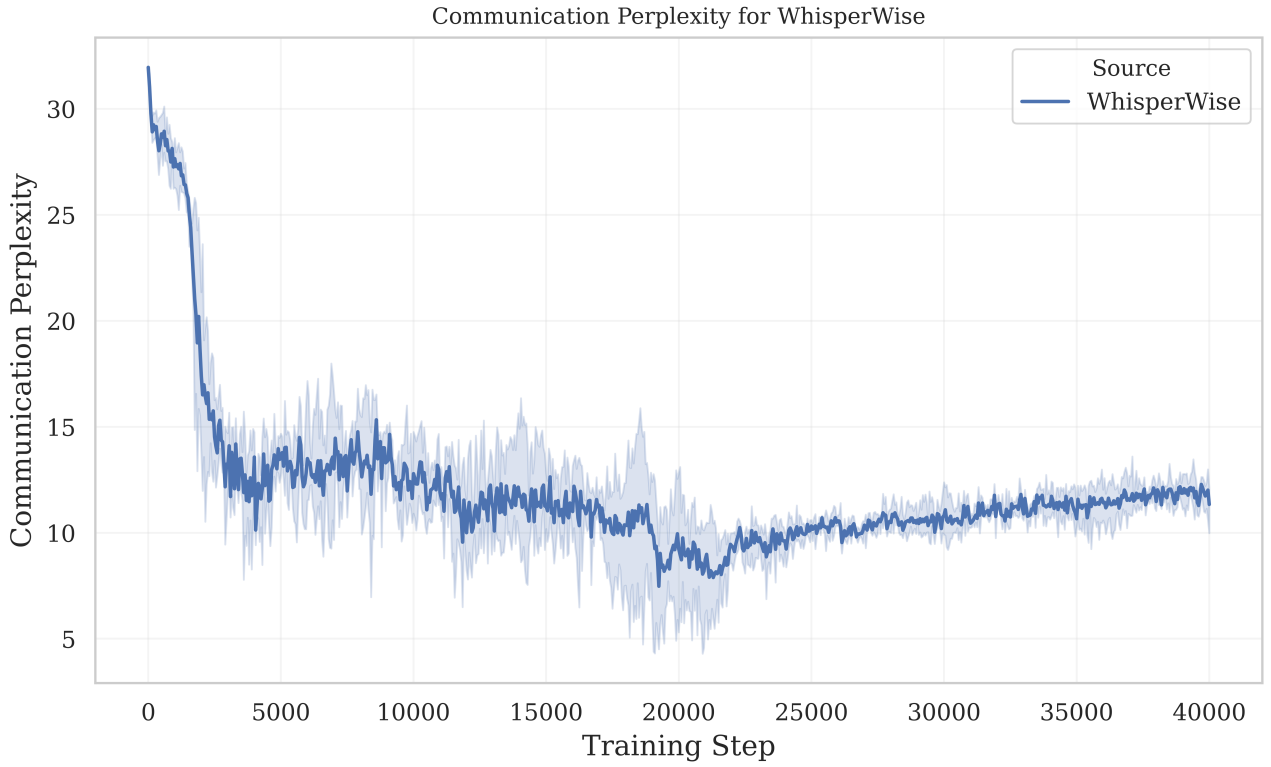


Figure G.3: Communication Perplexity tracked over a training run of WhisperWise trained across three runs. A 95% confidence interval is shown.

Appendix H

Example Configuration Files

Table H.1: Hyperparameters and simulation configuration.

Category	**Parameter**	**Value**
Communication	Communication type	aim
Communication	Vocabulary size	32
Communication	Messages per step	1
Communication	Autoencoder type	sq-vae
Communication	Residual quantization levels	1
Communication	Message embedding size	8
Communication	Comm. checkpoint folder	2026-04-23_20-22-07
AIM training	AIM learning rate	1×10^{-4}
AIM training	Observation runs	1
AIM training	AIM batch size	256
AIM training	Autoencoder epochs	25
AIM architecture	Hidden layer size	512
VQ-VAE	Commitment cost	0.25
SQ-VAE	KL weight	0.015
SQ-VAE	Min. temperature	0.05
SQ-VAE	Initial temperature	10.0
Imitation training	Autoencoder epochs	25
Imitation training	Observation runs	1
MAPPO	PPO epochs	4
MAPPO	Discount factor γ	0.99
MAPPO	GAE λ	0.95
MAPPO	Clipping coefficient	0.2
MAPPO	Value loss coefficient	0.5
MAPPO	Entropy coefficient	0.01
MAPPO	Actor learning rate	2×10^{-4}
MAPPO	Critic learning rate	1×10^{-4}
MAPPO architecture	LSTM hidden size	256
MAPPO architecture	Feature dimension	256
Training loop	Training steps	30000
Training loop	Simulation steps	30

Category	Parameter	Value
Training loop	Initial timestep	0
Training loop	Rollout phases	1
Training loop	PPO batch size	512
Training loop	Parallel worlds	960
Training loop	Seed	9
Training loop	Checkpoint interval	1000
Simulation env.	Arena width	5
Simulation env.	Arena height	5
Simulation env.	Headless mode	true
Simulation env.	Websocket URL	ws://127.0.0.1:3000
Simulation env.	Websocket path	ws
Simulation env.	Agent-agent visibility radius	10
Simulation env.	Agent-station visibility radius	10
Simulation env.	Initial agent layout	random
Simulation env.	Initial station layout	random

Appendix I

README

I.1 Emergent Communication in Multi-Agent Reinforcement Learning with Assembly Line Environments

Built for a Cambridge Computer Science undergraduate dissertation project.

This open-source program provides a framework for evaluation of emergent protocols in Multi-Agent Systems.

I.2 Getting Started

Before starting, initialise the virtual environment and build the simulation environment:

```
python -m venv .venv
.venv\Scripts\activate
pip install -e .
```

A set of eight configuration files are provided for customisability.

I.3 Running the program

The AI Mother Tongue communication protocol (AIM) requires pretraining a language. To train the language, run:

```
python -m controller.marl.main --mode train-language
```

Once the configuration files have been customised, training starts via:

```
python -m controller.marl.main --mode train
```

MARL often requires thousands or even millions of training steps to converge to an optimal policy. To accelerate this, imitation learning from an expert agent is used to pretrain the actor and critic. To use this, run:

```
python -m controller.marl.main --mode imitate
```

I.4 Customising Protocols

New communication protocols can be added to the folder `controller/marl/comms/`. The `comms.py` file contains an abstract class that new protocols should inherit from.

I.5 Analysis

A large set of evaluation tools are provided to analyse communication between agents. This includes:

- Discrete codebook usage
- Discrete codebook PCA
- Comparing training from multiple runs
- Memory probing
- Conditional probability of target or action given code

Appendix J

Glossary of Terms

Core Reinforcement Learning Concepts

RL *Reinforcement Learning*: A machine learning paradigm where agents learn to make sequential decisions by taking actions in an environment to maximise cumulative reward.

MARL *Multi-Agent Reinforcement Learning*: A subfield involving multiple agents situated in a shared environment, learning to maximise rewards through interaction. [2, 3]

MDP *Markov Decision Process*: A mathematical framework for modelling decision making in situations where outcomes are partly random and partly under the control of a decision maker.

POMDP *Partially Observable Markov Decision Process*: An extension of an MDP where the agent cannot directly observe the underlying state of the environment, requiring inference or communication. [4]

Dec-POMDP *Decentralised Partially Observable MDP*: A cooperative game framework where multiple agents act on the same state, receive individual local observations, and share a common reward signal. [8]

CTDE *Centralised Training Decentralised Execution*: A MARL paradigm where agents are trained using a centralised critic with access to global state information, but act independently during execution.

Reinforcement Learning Algorithms & Mechanisms

VDN *Value-Decomposition Networks*: A MARL architecture that assumes the joint action-value function can be additively decomposed into individual agents' value functions. [11]

TRPO *Trust Region Policy Optimisation*: A predecessor to PPO that updates policies by taking a step that improves performance while maintaining a strict region of trust. [16]

PPO *Proximal Policy Optimisation*: A policy gradient method that utilises a clipped surrogate objective function to ensure stable and conservative policy updates. [15, 7]

MAPPO *Multi-Agent PPO*: An extension of PPO tailored for multi-agent settings, utilising a shared actor and a centralised critic to mitigate non-stationarity. [14]

MADDPG *Multi-Agent Deep Deterministic Policy Gradient*: An off-policy actor-critic algorithm for MARL, noted as unsuitable for emergent communication due to semantic shifts. [13]

GAE *Generalised Advantage Estimation*: An algorithm used to calculate the advantage function, balancing the bias-variance trade-off by exponentially discounting future TD errors. [19, 49]

Autoencoders & Latent Representation Models

VAE *Variational Autoencoder*: A generative model architecture consisting of an encoder and a decoder, used to learn compressed, latent representations of data.

VQ-VAE *Vector-Quantised VAE*: An autoencoder that maps continuous latent vectors to the nearest discrete symbol in a learned finite codebook, acting as a semantic bottleneck. [26]

SQ-VAE *Stochastically-Quantised VAE*: A VAE variant using a variational Bayes framework that applies a categorical probability distribution over the codebook to mitigate collapse. [27]

vMF SQ-VAE *von Mises-Fisher SQ-VAE*: An extension of SQ-VAE that utilises cosine similarity and normalises embeddings onto a unit sphere. [27, 52]

HQ-VAE *Hierarchically-Quantised VAE*: A VAE variant that splits representations into hierarchical levels to capture macro-level and micro-level strategies. [28]

Multi-Agent Communication Protocols

DIAL *Differentiable Inter-Agent Learning*: A foundational multi-agent communication protocol that enables continuous gradient flow between agents during training. [21]

TarMAC *Targeted Multi-Agent Communication*: A framework that employs attention mechanisms to determine which agents to send continuous message vectors to. [22]

Neural Networks & Mathematical Components

MLP *Multi-Layer Perceptron*: A foundational feedforward artificial neural network architecture used to process inputs in actor and critic networks.

LSTM *Long Short-Term Memory*: A type of recurrent neural network module used to process sequences of data, enabling agents to retain allocentric and spatial memory.

STE *Straight-Through Estimator*: A mathematical technique to bypass non-differentiable operations (like argmin) by copying gradients directly from output to input during backpropagation.

PCA *Principal Component Analysis*: A dimensionality reduction technique used to project and visualise the latent language spaces and codebook embeddings.

MSE *Mean Squared Error*: A standard loss function that measures the average squared difference between a network's predicted values and the actual target values.

BCE *Binary Cross-Entropy*: A standard loss function used for binary classification or evaluating binary features during neural network training.

Whisperwise: Emergent Communication in Multi-Agent Reinforcement Learning for Complex Tasks

Part II Computer Science Project Proposal

May 13, 2026

Project Supervisor: **Ian Lewis**

Director of Studies: **Ramsey Faragher**

1.1 Introduction

Since Google DeepMind released AlphaGo in 2016, there has been a steady increase in interest in the field of reinforcement learning. This has led to breakthroughs in areas of finance, trading, robotics and even video games. A clear next step for reinforcement learning is multi-agent reinforcement learning, where multiple agents are placed into the same environment and have to learn to interact with one another efficiently. This area of research has the potential to improve surgery robots, traffic optimisations and search and rescue operations. However, there exists a range of challenges which make multi-agent reinforcement learning more difficult in practice, such as: non-stationary problem, credit assignment issues, and the one I will focus on in this project, communication problems. Unlike single-agent reinforcement learning, multi-agent reinforcement learning can communicate to improve the agent's knowledge of the environment and/or teach them the optimal next step. A widespread issue with communication is the "Joint Exploration Problem", which is a struggle for agents to communicate effectively in novel environments.

I propose to extend AI Mother Tongue[6] to a realistic situation for a more rigorous evaluation of the paper's proposed method. I will simulate an assembly line for multiple agents to evaluate the communication protocols. I will train the agents using a Value Decomposition Network to work together to explore the area. I will have a continuous communication protocol which broadcasts raw information from agent to agent and use this as a baseline to compare to my emergent communication protocol. This other protocol will use a vector

quantised variational autoencoder (VQ-VAE) to generate discrete symbols to broadcast from agent to agent.

This new method, using a VQ-VAE, allows communication protocols to emerge naturally rather than relying on manually designed rules or incentives. The limited alphabet forces agents to communicate with more compact, informative, and interpretable messages that prioritise essential information sharing to achieve coordination. Hence, we expect an improvement over the baseline communication protocol in the time taken to explore a given environment.

1.2 Structure of the Project

There are five main components to this project:

- Simulation
- Value Decomposition Network
- Communication Protocols
- Training
- Evaluation: refer to the Evaluation section

Simulation

The simulation is required as it will be how the reinforcement learning algorithm is both trained and evaluated. It will be responsible for the state management and local observations of each agent. Interactions will be handled through discrete timesteps, where agents produce actions informed by their policy and their communication inputs. As training multi-agent systems will require a lot of runs, the simulation must be optimised highly. It will be written in Rust for this reason and mainly function as a headless program (only providing a simple graphical UI when demoing single runs).

The simulation will involve agents working on an assembly line and communicating to pass items along the line.

Value Decomposition Network

The reinforcement learning will optimise the agent's policy (the agent's mapping from state space to action space). I have chosen to use a Value Decomposition Network to facilitate decentralised policy learning, as well as simultaneously optimising for a joint reward. Each agent will maintain its own Q function.

We want the agents to learn coordinated behaviour but maintain their own local observations and local policy. We also want to use a global reward function (successful items assembled). Due to these factors, I have chosen to use a Value

Decomposition Network (VDN). In VDN, the value function of the entire team, Q_{tot}, is factorised as the sum of the individual agents' value functions:

$$Q_{\text{tot}}(\tau, a) \;=\; \sum_{i} Q_i(\tau_i, a_i)$$

where Q_i is the action-value function for agent i, based only on its own action a_i and local action-observation history τ_i.

The VDN framework is particularly suitable for my project's cooperative multi-agent tasks:

- It naturally encourages cooperation by tying all agents' learning to the same joint reward.
- It is lightweight and efficient compared to more complex mixing architectures (like QMIX), allowing me more time to focus on other parts of the project.
- The decomposition aids in the interpretability of learned policies, as each agent's value estimates can be analysed independently.

Planned Implementation:

- Each agent will be equipped with its own Q-network, parameterised by a deep neural network, receiving as input both its local observations and any messages broadcast by other agents.
- The value decomposition will be enforced in the training loss, aligning each agent's updates to the global objective.
- This setup will also allow for seamless integration and fair comparison of different communication protocols, as the underlying learning dynamics for coordination are held constant.

Communication Protocols

This is the core of the project. I will primarily produce two distinct communication protocols. A simple protocol which has the goal of providing a baseline for the next protocol. It will emit raw continuous vectors. Our other protocol will emit symbols. These will be produced from a VQ-VAE (Vector Quantised - Variational Autoencoders), providing a discrete alphabet for agents to communicate via. Both of the protocols will initially broadcast every message to every other agent.

Training

The training will manage the model's optimisation process. It will use the simulation to collect data and update both the agents' Q functions and, when required, the VQ-VAE. I will add a simple logging and checkpointing framework to prevent loss of results.

1.3 Evaluation

Quantitive

I will gather a small subset of simulations not yet seen by the training, and run each model with the different communication protocols on each. I will then compare the average time for the agents to assemble all the items, as well as the count of successfully assembled items.

Qualitative

I will generate graphs of the distribution of the messages passed between agents. I will also observe the evaluation runs and note down strategies I see the agents come up with. I will then compare the two models' strategies.

1.4 Starting Point

I have limited experience in using reinforcement learning. However, I have previously built an autoencoder when analysing the stock market, which will help me develop the variational autoencoder. Furthermore, I have a small amount of experience using Rust from a three month internship I completed, but I have strong fluency in Python. This will help my development of a high-performance backend (Rust) and the ML (Python).

1.5 Success Criteria

1. Design and implement a simulation to train and demo multi-agent interactions. The simulation should be able to have single runs visualised. The simulation should control the state of every agent and collect data.

2. Implement a functional VDN-based multi-agent reinforcement learning (MARL) system capable of learning, given a simulation program with set methods.
3. Develop a communication protocol that allows agents to transmit raw data between each other.
4. Similarly, develop another communication protocol that uses a VQ-VAE to transmit vectors between agents.
5. The training and evaluation pipeline should be able to train and evaluate models automatically and allow interchangeably using both communication protocols.

1.6 Extensions

- Implement a third communication protocol using a hierarchically quantised variational autoencoder (HQ-VAE). This method was suggested by the AI Mother Tongue paper as a possible future work.
- Add hyperparameter tuning for my discrete communication protocol. This can involve the language size, quantisation levels, and message frequency to optimise agent coordination and learning efficiency.
- Add a new task where agents must solve for objectives other than an assembly line (e.g., coordinated navigation, resource gathering, or adversarial avoidance), enabling analysis of communication protocol effectiveness in more diverse MARL settings.

1.7 Timetable and Milestones

Weeks 1–2 (20 Oct 2025 – 2 Nov 2025)

- Set up Rust-based simulation environment using PyO3 to integrate with PyTorch.
- Prepare PyTorch environment for VDN.

Weeks 3–4 (3 Nov 2025 – 16 Nov 2025)

- Implement simulation for warehouse assembly line.
- Develop a wrapper for the simulation in Rust with py03 using an abstract class (to help potential later extension of using other tasks).

Weeks 5–6 (17 Nov 2025 – 30 Nov 2025): Core VDN Build

- Implement basic value decomposition networks (VDN).
- Develop basic training loop and agent coordination.
- *Milestone:* VDN architecture operational and agents learning basic tasks.

Weeks 7–8 (1 Dec 2025 – 14 Dec 2025)

- Integrate a continuous communication channel into the agent architecture using a modular design.
- Determine which data each agent should communicate.
- *Milestone:* Continuous communication protocol up and running.

Weeks 9–10 (15 Dec 2025 – 28 Dec 2025)

- Develop a variational autoencoder which quantises the latent space (VQ-VAE)
- Integrate the VQ-VAE into a discrete communication protocol for the agents.
- *Milestone:* Discrete communication protocol up and running.

Weeks 11–12 (29 Dec 2025 – 11 Jan 2026)

- Build a training pipeline supporting both communication protocols.
- Implement regular model checkpoints and experiment tracking.
- Automate logging and result collection for training runs.
- *Milestone:* Training pipeline working.

Weeks 13–14 (12 Jan 2026 – 25 Jan 2026)

- Run training on agents using both discrete and continuous communication protocols.
- Collate performance logs and monitor checkpointed models.
- *Milestone:* Complete initial fully trained models for both communication types.

Weeks 15–16 (26 Jan 2026 – 8 Feb 2026)

- Implement scripts to calculate key metrics: learning efficiency, coordination, bandwidth use.
- Automate model evaluation and result logging.

Weeks 17–18 (9 Feb 2026 – 22 Feb 2026)

- Start on extensions, focusing initially on implementing the alternative discrete communication protocol with an HQ-VAE.

Weeks 19–20 (23 Feb 2026 – 8 Mar 2026)

- Catch up on any uncompleted work or drafts.
- Any remaining time can be used to work on extensions.

Weeks 21–22 (9 Mar 2026 – 22 Mar 2026)

- Evaluate all models (continuous, discrete) and tested extensions on chosen environments.
- Collect and analyse results: success rate, sample efficiency, agent coordination, communication cost.

Weeks 23–24 (23 Mar 2026 – 5 Apr 2026)

- Catch up on any uncompleted work or drafts.
- Any remaining time can be used to work on extensions.

Weeks 25–26 (6 Apr 2026 – 19 Apr 2026)

- Write detailed chapters on implementation and evaluation.
- Create figures, tables, and result summaries.

Weeks 27–28 (20 Apr 2026 – 3 May 2026)

- Revise dissertation draft and submit for supervisor feedback.
- Edit dissertation based on feedback.

Week 29 (4 May 2026 – 17 May 2026)

- Final proofreading and formatting.
- Submit dissertation.
- Reserve leftover time for contingency: extra tests, additional evaluation, or further analysis if needed.
- *Milestone:* Submit dissertation and project code.

1.8 Resource Declaration

I will be using my personal desktop computer (AMD Ryzen 7 3700X, 32GB RAM, NVIDIA RTX 5060 Ti) as my primary machine for software development. As a backup, I will use my personal laptop and resources provided by SRCF (student-run computing facility). I will continuously back up my code and dissertation with Git version control on GitHub.